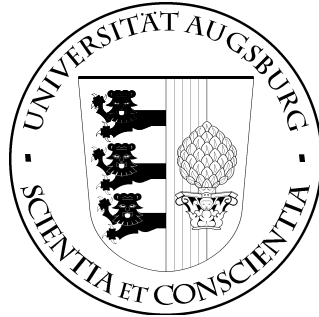


UNIVERSITÄT AUGSBURG
INSTITUT FÜR INFORMATIK



Bachelorarbeit im Studiengang Informatik

**Simulation eines Produktionsablaufes
mit AnyLogic**

Gutachter: Prof. Dr. Jörg Hähner
Zweitgutachter: Prof. Dr. Bernhard Bauer
Betreuer: Prof. Dr. Jörg Hähner
Datum: 23. Mai 2022

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	v
1 Vorwort	1
2 Abstract	1
3 Einleitung	2
3.1 Aufbau und Vorgehensweise	3
3.2 Zielsetzung der Arbeit	4
4 Grundlagen	5
4.1 Arten von Systemen	5
4.1.1 Zentrale Systeme	6
4.1.2 Dezentrale Systeme	7
4.1.3 Verteilte Systeme	9
4.2 Welches System ist das zu simulierende System	10
4.3 Simulator	11
5 Optimierung	13
5.1 Arten von Optimierungsansätzen	13
5.2 Stochastische Ansätze	15
5.2.1 Simulated Annealing	16
5.2.2 Tabu Search	16
5.3 Natur-inspirierte Optimierungsheuristiken	17
5.3.1 Evolutionäre Algorithmen	17
5.4 Metaheuristiken	18
5.5 Unification Ansätze	19
5.6 Auswahl Optimierungsansatz	19
6 Simulation	21
6.1 Vor- und Nachteile einer Simulation	21
6.2 Simulation eines realistischen Produktionsablaufes eines Unternehmens	23
6.2.1 Randdaten	25
6.3 Bedeutende Parameter für die Produktion	25
6.4 Ansätze des Unternehmens für die Optimierung des Produktionsablaufes/-flusses	26
6.5 Verfahrensweise	26
6.5.1 Umrisse des Unternehmens in Anylogic übernehmen	26
6.5.2 Einfügen der Maschinen	27

6.5.3	Erstellen von Blöcken (Blocks)	31
6.5.4	Statistiken	32
6.6	Erweiterung der Simulation anhand Java	33
6.6.1	Variablen	33
6.6.2	Funktionen	35
6.6.3	Parameter	37
6.6.4	Collections	37
6.7	Erfahrung	38
7	Auswertung	41
7.1	Aktueller Produktionsablauf	41
7.1.1	Analyse des aktuellen Produktionsablaufes	43
7.2	Optimierungsansätze von der Seite des Unternehmens	47
7.2.1	Analyse Simulation mit einem Zentrallager	48
7.3	Optimierung des Produktionsablaufes	50
7.3.1	Veränderungen an der Konfiguration	50
7.3.2	Veränderung der Reihenfolge der Aufträge	51
8	Zusammenfassung	55
9	Referenzen	57
A	Anhang	59
B	Inhalt der CD	60
C	Erklärung	61

Abbildungsverzeichnis

4.1	Graphische Darstellung eines zentralen Systems	7
4.2	Graphische Darstellung einer Peer-to-Peer Architektur [4]	8
4.3	Graphische Darstellungen dezentrales System	9
4.4	Graphische Darstellungen von verteilten Systemen	10
4.5	Umriss der Produktionsfläche mit Markierungen der Zwischenlager und Palettenregale	11
4.6	3D Darstellung eines Unternehmens mit AnyLogic	12
5.1	Wir haben in der Kurve drei Maxima. Das Ziel der Optimierung ist das Ermitteln des globalen Maxima. Dieser erzielt den höchsten Wert der Funktion f und ist somit die beste Systemkonfiguration	15
5.2	Chronologische Reihenfolge eines evolutionären Algorithmuses	17
5.3	Schnitt der Menge der Metaheuristiken mit der Menge der stochastischen Ansätze	18
6.1	Die Simulationssoftware Anylogic bietet die Möglichkeit Ressource Pools zu erstellen. Das sind wie Gruppen von Maschinen. Zum Beispiel haben wir ein Ressource Pool für die Lasermaschinen, Abkantpressen, CNC Maschinen oder Schweißerei. Daraufhin können Statistiken erstellt werden, die die Auslastung (Utilization) der jeweiligen Ressource Pools ausgeben.	23
6.2	Die Ausgabe von Auslastung von dem Lager und dem Stapler in Form von Statistiken. Zudem ist die prozentuale Auslastung zwischen Rohmaterialien und fertigen Teilen in dem Lager darstellbar. Da es sich in der Produktion größtenteils um schwere Materialien handelt, ist ein Stapler, mit dem Lastkraftwagen, eines der wichtigsten Fahrzeuge. Dieser sollte aufgrund seiner begrenzten Stückzahl so effizient wie möglich genutzt werden.	24
6.3	Agenten für die Simulation des Produktionsablaufes	28
6.4	Dreidimensionale Objekte, die Anylogic anbietet, die für die Simulation benötigt werden. Die Software hat Objekte in vielen verschiedenen Bereichen, weshalb es möglich ist für jede Art von Bereich die passenden Objekte zu finden.	28
6.5	Dreidimensionale Objekte für CNC Maschinen zu sehen, die für die Erstellung der Simulation benötigt werden	29
6.6	Netzwerk eines Produktionsablaufes der Simulation	30
6.7	Eingesetztes Netzwerk in die Simulation	30
6.8	Eingesetztes Netzwerk in die Simulation	32
6.9	Variablen Eigenschaften in AnyLogic	34
6.10	Datenbank	34
6.11	Laufzeitabfrage für das Laserschneiden	35
6.12	Logik der Simulation nach zwei Monaten	35
6.13	check() Funktion	36

6.14	Angelegte Collection für Abteilungen	36
6.15	<i>initializeDepartments()</i> Funktion	37
6.16	Pallet Racks im Netzwerk	39
6.17	Logik der Simulation	40
7.1	Draufsicht der ersten Produktionshalle	42
7.2	Draufsicht der zweiten Produktionshalle	43
7.3	Draufsicht des kompletten Firmengeländes	43
7.4	Echtzeitausgabe von Werten	44
7.5	Eingfügter Auftrag in die Datenbank	45
7.6	Auslastungen nach ungefähr 59 Stunden	45
7.7	Time Measure Start / - End für Zeitmessungen	46
7.8	Verteilung in den Abteilungen	47
7.9	Implementation des Zentrallagers	48
7.10	Verteilung der Auslastungen nach Einfügen eines Zentrallagers	48
7.11	Auslastung Simulation nach Einfügen eines Zentrallagers	49
7.12	Verteilung der Abteilungen nach minimaler Veränderung	50
7.13	Darstellung der Zeit für jeden Auftrag	52
7.14	Datenbank nach Veränderung der Reihenfolge	52
7.15	Darstellung der Zeit für jeden Auftrag nach Veränderung der Reihenfolge	53
7.16	Gesamtlaufzeit zufällig generierter Reihenfolge: 220.391 Sekunden	53
7.17	Gesamtlaufzeit zufällig generierter Reihenfolge: 216.614 Sekunden	54
7.18	Gesamtlaufzeit zufällig generierter Reihenfolge: 206.891 Sekunden	54

Tabellenverzeichnis

5.1	Optmierung Probleme [8]	13
-----	-------------------------	----

1 Vorwort

Die vorliegende Bachelorarbeit behandelt die Simulation und die Optimierung eines Produktionsablaufes. Diese wird mit der Simulationssoftware AnyLogic erstellt. Dabei wurde die Forschung der Simulation von einem Unternehmen mit realen Aufträgen durchgeführt. Diese Bachelorarbeit verfasste ich als Abschlussarbeit meines Informatikstudiums an der Universität Augsburg.

Das Ziel der Arbeit war die Erstellung realer Produktionsabläufe in einer Simulation und diese dementsprechend zu beschleunigen. Anfangs beschäftigte ich mich mit der Übernahme der Daten eines Unternehmens in die Simulation. Diese beinhalten die Umrisse, Anlieferung, Auslieferung und die Positionierung und Laufzeit der jeweiligen Maschinen.

Hiermit geht eine besondere Danksagung an meinen Betreuer Prof. Dr. Jörg Hähner, der mich während der Arbeit stets unterstützt hat. Zusammen mit Prof. Dr. Jörg Hähner wurde das Thema für die Bachelorarbeit erstellt. Durch seine Fachkenntnisse im Bereich der Informatik und Organic Computing wurden mir wertvolle Einblicke verschafft. Außerdem waren für mich die Vorlesungen, die ich bei Prof. Dr. Hähner besuchen durfte und die langjährige Erfahrung, die mir Prof. Dr. Hähner weitervermittelt hat, sehr hilfreich.

Die Forschung und die Erstellung der Simulation wies einige Schwierigkeiten auf, doch nach einer Einarbeitungsphase und ausreichender Recherche gelang es mir, diese Probleme zu beheben.

Ein weiterer wesentlicher Dank geht an den Geschäftsführer der Firma Stern Metallbau. Dieser hat alle benötigten Daten und Pläne für die Erstellung der Simulation zur Verfügung gestellt.

Schlussendlich geht auch eine herzliche Danksagung an meine Familie, ohne deren Unterstützung mein Studium kaum möglich gewesen wäre.

Augsburg, 23.Mai.2022

2 Abstract

Simulationen nehmen in der heutigen Zeit eine große Rolle ein, da diese es ermöglichen reale Probleme zu übernehmen und diese zu simulieren, zu testen und zu verändern. Diese wurde mit der Simulationssoftware AnyLogic erstellt. Mithilfe der Verknüpfung der Simulation mit der Programmiersprache Java ist es möglich komplexe Simulationen zu erstellen. Für die Erstellung der Simulation war enorm viel Vorarbeit notwendig. Vor der Erstellung der eigentlichen Simulation wurden die Tutorials, die von der Simulationssoftware AnyLogic bereitgestellt wurde, durchgeführt. Nach der Einarbeitung anhand der Tutorials begann die Planung für die Simulation. Dies erfolgt anhand der Erstellung der Umrisse der Geländes, der Hallen und des Bürogebäudes.

Das Thema für diese Arbeit entstand nach Absprache mit dem Betreuer Prof. Dr. Jörg Hähner. Die Anwendung von Simulationen für die Analyse von Produktionssystemen bietet einige Vorteile. Eines dieser wesentlichen Vorteile ist das Testen von verschiedenen Konfigurationen des Systems. Dadurch besteht die Möglichkeit, ob die durchgeführten Veränderungen in diesem System zu einer Verbesserung beitragen. Durch das Testen der verschiedenen Konfigurationen, ohne diese direkt in der Produktion anzuwenden, wird das Risiko einer Verschlechterung deutlich verringert.

Während der Erstellung der Simulation traten immer wieder Schwierigkeiten auf. Im Laufe der Zeit wurden diese stets gelöst, jedoch waren diese Fehler und Probleme sehr zeitaufwendig, da das Simulationsframework komplex ist und das Lösen der Fehler meist nicht einfach war. Hinzu kommt, dass die zu Beginn durchgeführten Tutorials einfach und simpel gehalten worden sind, um die Basics zu verstehen und ein Grundverständnis für das Framework zu generieren. Jedoch ist das zu simulierende System komplexer und deutlich größer als die zur Verfügung gestellten Beispiele.

Bei der Erstellung der Simulation wurde abschnittsweise vorgegangen. Zuerst wurden Objekte und Gebäude eingefügt. Danach wurden die benötigten Agenten erstellt. Die erste Simulation wurde einfach gehalten und war ein einfacher Auftrag von einem Produkt, das lediglich drei Arbeitsschritte benötigte. Die dafür benötigte Logik und die Zeit für die dafür benötigten Arbeitsschritte wurden statisch festgelegt. Im späteren Verlauf wurden Funktionen, Variablen, Collections, Parameter und eine Datenbank angelegt, um eine realistische Simulation zu programmieren.

Für die Simulation wurden reale Aufträge des Unternehmens in die Datenbank übernommen, simuliert und analysiert. Das Ziel dieser Arbeit ist die Erstellung und Analyse eines Produktionsablaufes mit dem Simulationsframework und die Optimierung des Produktionsflusses, anhand der Veränderung der Simulation.

3 Einleitung

Heutzutage ist eine effiziente und schnelle Produktionsabfertigung ausschlaggebend für die Konkurrenzfähigkeit eines Unternehmens. Das Optimieren der Abläufe in der Produktion bietet einem Unternehmen deutliche Vorteile gegenüber der Konkurrenz und trägt zu einer effizienteren Abfertigung bei. Das Lösen von Problemen innerhalb des Systems und die Optimierung von Produktions- und Logistiksystemen ist durch die Erstellung von Simulationen möglich. Die Anwendung von Simulationen weisen einige Vorteile auf, weshalb die Anwendung von Simulationen auf großes Interesse stoßen. Die Vorteile werden in dieser Arbeit genannt und anhand von Beispielen verdeutlicht. Diese werden im sechsten Kapitel genauer betrachtet. Für die Simulation eines Produktionsablaufes wurde das Unternehmen STERN Metallbau verwendet. Das Unternehmen lieferte Grundrisse, Positionen von Maschinen und reale Aufträge mit den jeweiligen Daten und Arbeitsschritten. Wie dieses Unternehmen in die Simulation übernommen wird, wird ebenfalls in Kapitel sechs erläutert.

Simulationen werden seit mehreren Jahren für die Planung von Systemen im Bereich der Produktion und Logistik eingesetzt. Zudem ist die Analyse von Produktionsabläufen anhand der Darstellung von Diagrammen, die in Echtzeit erstellt werden, möglich. Das Ziel einer Simulation sind Verbesserungen der Laufzeit und der Abwicklung in der Produktion. Anhand einer dreidimensionalen Simulation besteht die Möglichkeit für den Nutzer visuell Probleme wahrzunehmen und zu beheben. Die Behebung dieser Probleme kann anhand der Veränderung von Parametern oder Variablen erfolgen. Führt die Veränderung des Ablaufes nach einigen Testversuchen zu einer Optimierung des Prozesses und einer Verbesserung der Laufzeit, so besteht die Möglichkeit, dass das Unternehmen die durchgeführte Veränderung übernimmt und somit einen effizienteren Produktionsablauf gewährleisten kann.

Simulationen werden heutzutage in vielen Bereichen eingesetzt. Demnach werden Simulationen für die Automobilindustrie, Verkehrssimulation, Flugsimulation, Robotik und Spiele verwendet. Der Nutzen von Simulationen bietet einige Vorteile, die in der Marktwirtschaft eine große Rolle spielen. Angenommen, dass ein Unternehmen einige Optimierungsansätze hat und diese gerne in ihrer Produktion anwenden wollen würde. Jedoch ist das Problem, dass die Umsetzung in den meisten Fällen sehr kostenintensiv ist. Wenn die Veränderung in den Produktionsablauf implementiert werden soll, so muss in den meisten Fällen die Produktion angehalten werden, um die Veränderungen zu realisieren. Bei einem Produktionsstopp ist mit zusätzlichen Kosten und einem zeitlichen Aufwand zu rechnen, aufgrund von Optimierungsansätzen, die zuvor nicht getestet worden sind. Der Hauptaspekt für die Verwendung von Simulationen ist, dass diese vor der Abgabe getestet werden und somit vor der Publizierung keinen Schaden in Form von Fehlern oder Verschlechterungen, die während der Betriebsnahme, anrichten können.

3.1 Aufbau und Vorgehensweise

In diesem Abschnitt handelt es sich um den Aufbau und die Vorgehensweise der Bachelorarbeit. Für die Erstellung der Simulation werden zwei Werkzeuge benötigt. Das Erste ist die Programmiersprache Java und das zweite Werkzeug ist die Simulationssoftware. Da Erfahrung in der Programmiersprache bestehen, musste ich mich intensiv in die Simulationssoftware AnyLogic einarbeiten. Dies geschah anhand der Erstellung von Simulationen in den verschiedensten Themengebieten, damit ein Grundwissen für die Simulation und *Process Modelling Library* erlernt werden konnte. Schon bei der Erstellung dieser Simulationen entstanden Probleme. Denn die Simulationssoftware bietet eine große Vielfalt von Möglichkeiten, um eine Simulation zu erstellen. Im Laufe der Arbeit fiel immer wieder ein Nachteil auf, welches oft zu Problemen fuhr. Das Problem ist die geringe Anzahl der Community. Wenn man zum Beispiel ein Java, Python, oder C Programm schreibt und dieser einen Fehler wirft, so findet man in wenigen Sekunden mehrere hilfreiche Antworten. Die Community (online) von Entwicklern, die die Simulationssoftware AnyLogic verwenden, ist nicht groß. So mussten bei Fehlern und Errors verschiedene Methoden und Blocks getestet werden, um zum Beispiel Agenten von einer Abteilung in die andere zu schicken.

Jedoch ist ein großer Vorteil von AnyLogic, dass dies ein Java basiertes Framework ist und die Möglichkeit besteht Funktionen, Variablen und Methoden zu verwenden. Außerdem ist dieser Aspekt ein Grund für die Erstellung von komplexen Simulationen.

Der Aufbau der Bachelorarbeit ist in mehrere Kapitel unterteilt. Zu Beginn wurde im vorigen Abschnitt die Motivation für das ausgewählte Thema erklärt. Die Auswahl der Thematik wurde zusammen mit dem Prof. Dr. Jörg Hähner selektiert und im Laufe der Zeit spezifiziert. Im vierten Kapitel werden die Grundlagen definiert, die für die Erstellung und Analyse eines Produktionsablaufes benötigt werden. Zudem werden in diesem Kapitel die drei verschiedenen Systeme erläutert. Anhand von Definitionen, Merkmalen und Beispielen der jeweiligen Systeme wurde entschieden, um welches System es sich in dem vorliegenden Produktionsablauf handelt. In dem selben Abschnitt wird das Simulationsframework vorgestellt und die Vorteile dieses Frameworks genannt. In dem darauffolgenden Kapitel handelt es sich um die verschiedenen Optimierungsansätze. Diese spalten sich auf in stochastische, natur-inspirierte Optimierungsheuristiken, Metaheuristiken und Vereinigungsansätze (Unification Ansätze). Außerdem werden zu den jeweiligen Ansätzen, die dafür bekanntesten Methoden erklärt. Anschließend wird erläutert, weshalb die Auswahl von Metaheuristiken am besten ist.

In Kapitel sechs wird auf die Simulation eingegangen. Darin handelt es sich im Allgemeinen um die Vorteile und auch Nachteile von der Anwendung von Simulationen. Daraufhin wird der zu simulierende Produktionsablauf geschildert und graphisch dargestellt, wobei Randdaten zum simulierenden Unternehmen genannt werden. Außerdem werden Ansätze seitens des Unternehmens verwirklicht in der Simulation, um zu testen, ob diese zu einer Verbesserung des Produktionsablaufes führen.

Im siebten Kapitel werden die Simulationen anhand von Statistiken ausgewertet und verglichen. Daraus lässt sich erschließen, wie effizient der Produktionsablauf ist und welche Ansätze zu einer Verbesserung der Simulation führen. Schließlich wird im letzten Kapitel eine Zusammenfassung verfasst, welche die komplette Arbeit und Ergebnisse aufzeigt und die Erfahrungen mit der Simulationssoftware widerspiegelt.

3.2 Zielsetzung der Arbeit

Das Ziel der Arbeit ist die Simulation eines Produktionsablaufes mit der Simulationssoftware AnyLogic. Das dafür verwendete Unternehmen wird mit dem Java basierten Simulationsframework AnyLogic simuliert. Für die Erstellung der Simulation fanden mehrere Treffen mit dem Geschäftsführer des Unternehmens statt. Dieser stellte die Frage, ob es möglich sei Optimierungsansätze zu implementieren und zu testen, um eine Optimierung des Ablaufes zu generieren, die Effizienz zu steigern und wichtige Eigenschaften und Parameter zu verbessern. Parameter, die für die Produktion eine wichtige Rolle spielen, werden im Abschnitt 6.3 genannt und zudem die Gründe für deren Bedeutung. Da vielen Unternehmen eine begrenzte Anzahl an Ressourcen zur Verfügung stehen, sei es der Firmenplatz, die Arbeiter, das Geld und die Maschinen, ist eine optimale Konfiguration des Produktionsablaufes die einzige Lösung, um eine Wettbewerbsfähigkeit zu gewährleisten.

4 Grundlagen

Im folgenden Kapitel handelt es sich um die Grundlagen für die Simulation und Analyse von Produktionssystemen. Eine Simulation ist eine Operation an dem Model des Systems. Anhand des Models ist es möglich Veränderungen und Modifikationen im System durchzuführen. Der weitestgehende Grund für die Nutzung ist die Bewertung der Leistung eines Systems in unterschiedlichen Konfigurationen über einen längeren Zeitraum in Echtzeit. [12]

Für die Erstellung der Simulation sind Kenntnisse mit der Simulationssoftware AnyLogic eine Voraussetzung. Die Umsetzung des Systems in eine Simulation anhand von gegebenen Methoden, Blöcken, Netzwerken und Agenten müssen korrekt angewendet werden, denn diese sind zu vergleichen mit einem Java Programm. Wird einem Objekt falsche Parameter oder Methoden übergeben, so wird das Programm abgebrochen und die Software ist nicht in der Lage die Simulation zu starten.

Anhand der Programmiersprache Java ist es möglich die Simulation mit dem Code zu verknüpfen. Dies öffnet neue Möglichkeiten, um die Simulation zu erweitern und komplexe Systeme zu erstellen. Kenntnisse in einer Programmiersprache, wie hier zum Beispiel Java, sind für die Erstellung einer komplexen Simulation in AnyLogic eine Voraussetzung, da AnyLogic ein Java basiertes Framework ist.

Zudem ist es möglich Fehler und Bugs anhand der Main.java Klasse zu erkennen und zu beheben. Die Main.java Klasse steuert die komplette Simulation. Sollte es beim Starten der Simulation zu Fehlern kommen, besteht die Möglichkeit die Fehler im Programm zu finden und diese anschließend zu beheben, unter der Bedingung, dass der Nutzer die Programmiersprache Java beherrscht. Zudem besteht die Möglichkeit Diagramme, Statistiken, dreidimensionale Objekte, Parameter und Variablen, die zu einem späteren Zeitpunkt verändert werden können, anzulegen.

4.1 Arten von Systemen

Ein System ist ein ganzheitlicher Zusammenhang von Einheiten, deren Beziehung untereinander sich quantitativ und qualitativ von ihren Beziehungen zu anderen Entitäten abheben.[19] Jedes Unternehmen besitzt ein System, welches dazu dient Produktionsabläufe schnellst- und bestmöglich abzuarbeiten. Es wird zwischen drei Systemen unterschieden. Diese werden eingeteilt in zentrale, dezentrale und verteilte Systeme.

Jedes System hat seine Vor- und Nachteile. In den nächsten Abschnitten werden die unterschiedlichen Systeme verglichen und genauer erläutert. Zudem werden die Einsatzgebiete und verschiedenen Architekturen der jeweiligen Systeme genannt.

4.1.1 Zentrale Systeme

Beginnend mit den zentralen Systemen, da diese die einfachste Form von Systemen sind. Aufgrund des vorliegenden Produktionsablaufes kann ein zentrales System anhand eines Lagersystems erklärt werden. Auf die Produktion verteilt sind ein oder mehrere kleine Teillager vorhanden. Diese sind direkt verbunden mit einem zentralen Lager. Alle fertigen Produktionsteile werden am Ende der Produktion in das Zentrallager gebracht und von diesem aus werden die produzierten Teile aufgeladen und ausgeliefert. Das Zentrallager organisiert wichtige Parameter, wie zum Beispiel den aktuellen Warenbestand.

Zudem weisen zentrale Systeme Charakteristiken auf. Diese sind, dass in einem zentral Lager ein oder mehrere Teillager vorhanden sind, die von dem zentralen Lager synchronisiert werden.

Eine weitere Eigenschaft eines zentralen Systems ist das Vorhandensein eines einzigen Knotens/Nodes, der alle anderen Knoten steuert. Jedoch weist ein zentrales System eine große Problematik auf. Die entsteht, wenn es zum Versagen des zentralen Knotens kommt, aufgrund von Verbindungsproblemen und diese nicht mehr miteinander kommunizieren können. Dieses Problem ist deutlich zu erkennen anhand der Betrachtung der *Abbildung 4.1*. Bei der Annahme, dass der zentrale Knoten ausfällt, sind die anderen Knoten/Teillager nicht mehr in der Lage miteinander zu kommunizieren.

Jedoch bietet ein zentrales System auch einige Vorteile. Schnelle Updates sind anhand des zentralen Knotens möglich, indem man sich diesen wie einen Zentralrechner/Server vorstellt. Hinzu kommt, dass das Sichern und Warten der Knoten vereinfacht wird. Ein weiterer Aspekt ist die Anpassungsfähigkeit des Systems. Durch das Entfernen und Hinzufügen von Knoten kann das System modifiziert werden. Soll zum Beispiel ein Knoten von dem zentralen System entfernt werden, ist es möglich die Verbindung zum Zentralknoten zu trennen und somit den jeweiligen Knoten vom System zu trennen.

Ein Nachteil von zentralen Systemen ist die Abhängigkeit der Konnektivität. Durch Auflösung der Verbindung zwischen Knoten und Zentralknoten bricht die Kommunikation zusammen und das ganze System steht still. Ein weiterer Nachteil ist die Instandhaltung des Zentralknotens/Servers, da aufgrund dessen die Knoten wiederum nicht arbeiten können. Sollte es zudem zu einem Problem des Zentralknotens kommen und es wurde in der Vergangenheit kein Backup, weder physisch noch cloud-basiert, erstellt, so hat das Unternehmen ein maßgebliches Problem mit der Wiederherstellung der Daten.

Anwendung finden zentrale Systeme vor allem in Sensornetzen. Auf einer größeren Fläche befinden sich mehrere Sensoren, die leistungsschwächer sind, da diese dazu dienen, die erfassten Daten an den Zentralknoten oder an den Server weiter zu leiten. Eine weitere Anwendung ist eine zentrale Datenbank. Ein System, dass daraus besteht alle Daten in den Server oder auf dem Zentralcomputer zu speichern. Meist werden zentrale Systeme in kleineren Räumlichkeiten oder Unternehmen verwendet, da diese kosteneffizienter umzusetzen sind. [1]

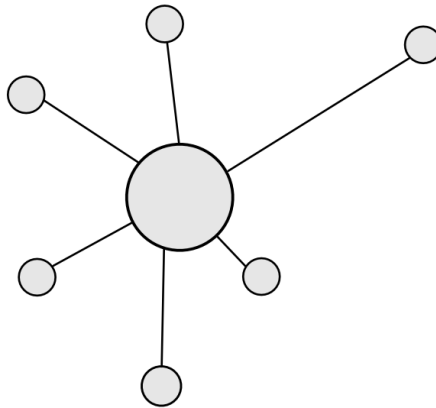


Abbildung 4.1: Graphische Darstellung eines zentralen Systems

4.1.2 Dezentrale Systeme

Eine weitere Art von Systemen sind die dezentralen Systeme. Das besteht meist aus gleichwertigen Komponenten/Knoten und einer Verbindung. In Gegensatz zu zentralen Systemen weist ein dezentrales System meist leistungsstärkere Komponenten auf. Dies liegt an der Ursache, dass die keinen zentralen Knoten haben, der diese steuert, synchronisiert und kontrolliert. Aufgrund dessen ist das Verhalten von Komponenten innerhalb eines dezentralen Systems deutlich komplexer. Jede/r Knoten/Komponente in diesem System treffen ihre eigenen Entscheidungen.

Eine grafische Darstellung für ein dezentralisiertes System ist in der *Abbildung 4.1* dargestellt. Das bekannteste Beispiel für eine dezentrales System ist Bitcoin. Bitcoin ist das bekannteste dezentrale Buchungssystem, da keine Regierung oder Organisation dieses System besitzt.

In dezentralen Systemen gibt es eine Aufspaltung in zwei Architekturen. Dies ist die peer-to-peer (p2p) und master-slave Architektur. In der *Abbildung 4.2* ist eine einer peer-to-peer Architektur zu sehen. Diese weist einige Merkmale auf. Diese wären unter anderem die Gleichberechtigung der Komponenten innerhalb des Systems. Ein weiteres Merkmal dafür ist die Autonomie und selbstständige Organisation innerhalb des Systems. Dies ist ein Grund für die Niveaudifferenz der Leistung der jeweiligen Komponenten.

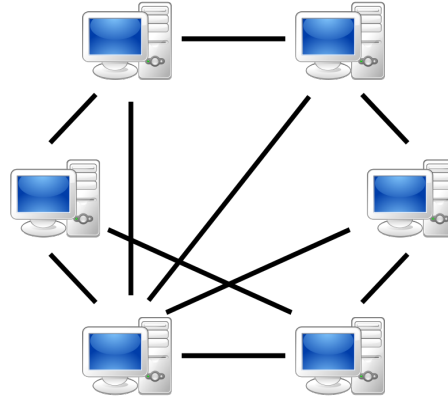


Abbildung 4.2: Graphische Darstellung einer Peer-to-Peer Architektur [4]

In der Master/Slave Architektur sind die Komponenten in Masters (Meister) und Slaves (Sklave) unterteilt. An dieser Unterteilung ist deutlich zu erkennen, dass es sich in dieser Architektur um eine hierarchische Verwaltung auf eine gemeinsame Ressource (in Form von einem Datenkanal) innerhalb eines Systems handelt. Der Master hat die Kontrolle im System und steuert mindestens einen oder mehrere Slaves. Das Zugreifen auf Ressourcen ist den Slaves ohne Erlaubnis des Masters nicht gestattet. Der Master übernimmt zudem die Adressierung der Slaves. Anhand dieser ist es dem Master möglich den Slave über eine Verbindung anzusprechen. [3]

Die Eigenschaften eines dezentralen Systems ist der Mangel an Synchronisation, aufgrund des fehlenden Hauptknotens, welcher alle anderen Knoten in regelmäßigen Abständen synchronisiert, um zeittechnische Probleme zu vermeiden. Vorteilhaft gegenüber zentralen Systemen ist der Aspekt, dass das Ausschalten/Ausfallen des Hauptknotens nicht zu einem totalen Ausfall führen würde. Der Ausfall eines Knotens würde lediglich zum Ausfall eines Teiles des Systems führen.

Zudem ist es möglich einzelne Komponenten zu optimieren, hinsichtlich der Soft-/ Hardware, um die Leistung des Systems zu optimieren. Angenommen wir haben in der vorliegenden Produktion alte Stapler, welche aufgrund von Ausfällen oder geringer Geschwindigkeiten den Produktionsablauf verlangsamen. Die Optimierung wäre dann zum Beispiel ein Fahrzeug zu kaufen, das eine höhere Geschwindigkeit erreichen würde und weniger anfällig für Ausfälle wäre. Die *Abbildung 4.3* zeigt die mögliche Konfiguration von dezentralen Systemen, anhand von Kanten und Knoten. Dezentrale Systeme finden meist Anwendung in privaten Netzwerken, peer Knoten können integriert werden und somit das private Netzwerk erweitern. Ein weiteres Anwendungsbeispiel ist eine dezentralisierte Datenbank. Bei unserem Anwendungsbeispiel können wir unser Lager, welches eine Datenbank darstellt, in mehrere Abteilungen unterteilen. Die Storage und Nutzung dieser wird auf verschiedene Knoten verteilt.

Die Anwendung von dezentralen Systemen finden sich in Kryptowährungen und auch Ad-hoc Netzen (MANets). Ad-hoc Netze sind eine dezentrale Art von drahtlosen Netzwerken. Genutzt werden dezentrale Systeme auch unter anderem von Bitcoin und dem Tor Netzwerk. [1]

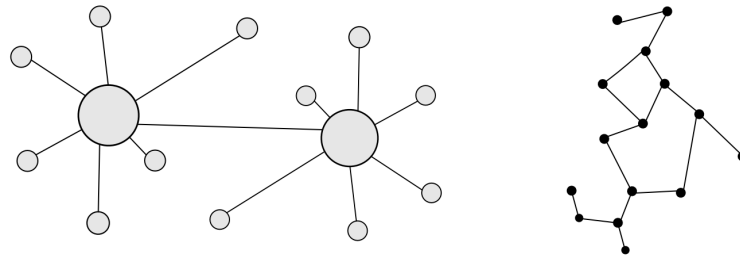


Abbildung 4.3: Graphische Darstellungen dezentrales System

4.1.3 Verteilte Systeme

Ein verteiltes System (distributed system) ist die dritte Art von Systemen. Jede Knoten in dem System trifft eigene Entscheidungen. Jedoch wird die finale Handlung des Systems von der Aggregation jedes Knotens entschieden. Es gibt keinen Knoten, der das Empfangen und Verteilen von Anfragen bestimmt. In *Abbildung 4.4* ist eine graphische Darstellung aus Knoten (Komponenten) und Kanten (Verbindung) von einem verteilten System. Die Skalierung dieser verläuft sowohl horizontal als auch vertikal.

Wie in den dezentralen Systemen gibt es in den verteilten Systemen verschiedene Arten von Architekturen. Diese sind die peer-to-peer, client-server und n-tier Architekturen. Die Erläuterung der peer-to-peer und client-server Architekturen finden sich im vorherigen Abschnitt *Dezentrale Systeme*. Die n-tier oder Schichtenarchitektur kennzeichnet sich durch die Verteilung verschiedener Teile einer Anwendung auf verschiedene Knoten im System. Anhand der Zusammenarbeit der Knoten können diese als Anwendung für den Benutzer funktionieren.

Jedoch weisen diese Art von Systemen Einschränkungen auf. Die Entwicklung und Fehlersuche für Systeme ist komplexer als in zentralen und dezentralen Systemen. Deshalb steigt dadurch auch die Komplexität dieser Algorithmen mit der Anzahl an Komponenten im System. Das Fehlen einer gemeinsamen Uhr führt zu der Problematik bei der zeitlichen Abfolge von Ereignissen und Transaktionen. [1]

Dies ist wiederum ein Grund für die Wichtigkeit der Zeitsynchronisation von Komponenten innerhalb eines Systems. Gründe dafür sind Offsets der Uhren, Oszillatoren weisen zufällige Abweichungen von der Nennfrequenz oder auch Taktdrift (clock drift). Für das Lösen von Problemen von Zeitsynchronisation gibt es einige Ansätze, wie die Zeitsynchronisation von Römer, Reference Broadcast Synchronisation, Flooding Time Synchronisation Protocol oder der Reachback Firefly Algorithm. [20]

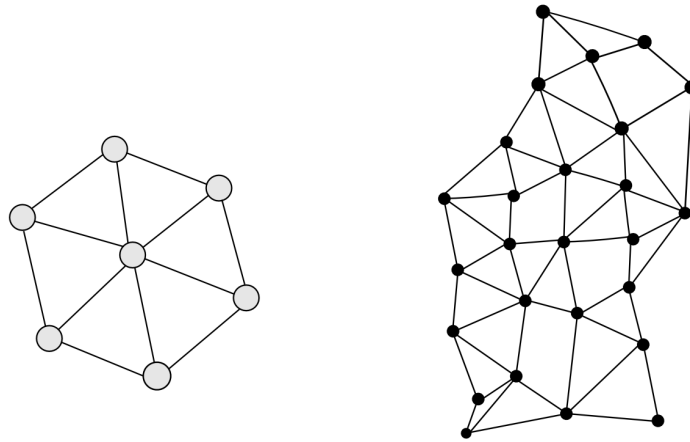


Abbildung 4.4: Graphische Darstellungen von verteilten Systemen

4.2 Welches System ist das zu simulierende System

Im folgenden Abschnitt ist die Frage zu beantworten, welches System das vorliegende Unternehmen anwendet. Es ist deutlich zu erkennen, dass es sich hierbei nicht um ein zentrales System handelt, da Waren innerhalb des Unternehmens verlagert werden. In den Hallen sind jeweils ein Wareneingang und -ausgang enthalten und es ist kein Zentrallager vorhanden, in dem die Waren gelagert werden. Ein verteiltes System ist es auch nicht, da es sich in dem Produktionsablauf um nicht autonome Komponenten handelt und diese selber keine Entscheidungen treffen können.

In *Abbildung 3.5* ist der Umriss der Produktionshallen zu sehen. Deutlich zu erkennen, dass es sich hierbei um zwei Produktionshallen handelt und die erste Halle auf der rechten Seite in drei kleinere Hallen eingeteilt ist. In beiden Hallen sind Palettenregale und Zwischenlager gekennzeichnet.

Die in der Abteilung gefertigten oder bearbeiteten Waren werden in den jeweiligen Lager (hier: in Rot) gelagert und dann in das nächste Lager gebracht und von dort aus werden wieder die nächsten Arbeitsschritte abgearbeitet, bis das fertige Produkt gefertigt wurde. Daran ist deutlich zu erkennen, dass es sich hierbei um ein dezentrales Lagersystem handelt.

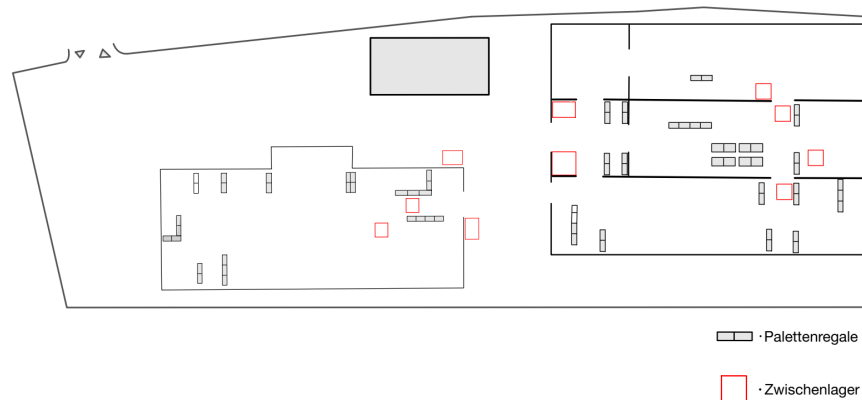


Abbildung 4.5: Umriss der Produktionsfläche mit Markierungen der Zwischenlager und Palettenregale

4.3 Simulator

In dem letzten Abschnitt des Kapitels wird die für die Simulation verwendete Software verwendet. Aktuell sind einige Simulationsprogramme auf dem Markt vorhanden. Für die Simulation des Produktionsablaufes wird die Simulationssoftware AnyLogic eingesetzt. AnyLogic ist ein Simulationswerkzeug von dem Unternehmen The AnyLogic Company. Die Software unterstützt systemdynamische, ereignisorganisierte und agentenbasierte Simulationsmethodiken. Diese drei Methoden können beliebig miteinander kombiniert werden, um Systeme und Prozesse beliebiger Komplexität zu simulieren. In AnyLogic können verschiedene grafische Modellierungsarten verwendet werden. AnyLogic ist das erste Werkzeug, das Multimethodenmodellierung eingeführt hat und ist nach wie vor die einzige Software, die diese Fähigkeit besitzt. Ausschlaggebend für die Auswahl von Anylogic ist die Erweiterbarkeit von Modellen anhand der Programmiersprache Java. Diese ermöglichen dem Nutzer jegliche Modellierungsmöglichkeiten. Die Darstellung von zwei- und dreidimensionalen Objekten ist beachtenswert und komplexe Produktionsdarstellungen sind möglich. [11]

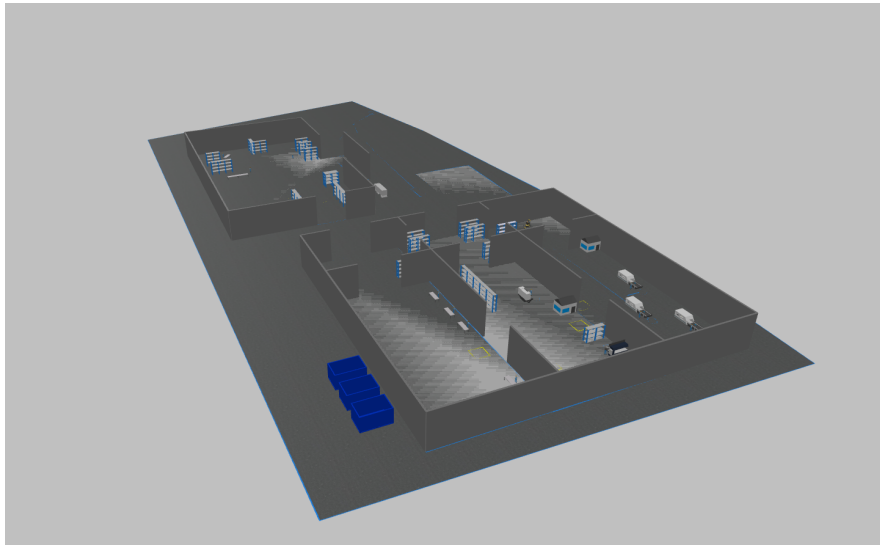


Abbildung 4.6: 3D Darstellung eines Unternehmens mit AnyLogic

5 Optimierung

Die Optimierung spielt in jeder Hinsicht eine wichtige Rolle, sei es in der Wirtschaft oder im Ingenieurwesen. Das Ziel des Menschen immer die bestmögliche Lösung zu finden spielt immer mehr eine große Rolle. Die Theorie, Ansätze und Methoden der Optimierung behandeln das Auswählen der bestmöglichen Alternative.

Der Bereich der Optimierung gewinnt immer mehr an Bedeutung. Der Grund für die Bedeutung ist die steigende Knappheit von Ressourcen und der immer größer werdende Konkurrenzkampf. Die schnelle Abwicklung von Produkten und Beschleunigung des Produktionsflusses wird immer elementarer.

Für die Optimierung gibt es verschiedene Ansätze, um eine Optimierung in jedem Bereich zu gewährleisten. In Bereichen der Informatik, Produktion, Lieferverkehr, Planung oder Bau von Immobilien. Für jedes Problem gibt es eine Optimierung. Angenommen wir nehmen den Bereich der Informatik und es handelt sich um Programm, welches etwas berechnen muss, jedoch ist die Laufzeit des Programms aufgrund des rekursiven Aufrufs einer Methode zu lang. Ersetzt man diese Methode durch eine iterative Methode zur Berechnung, ist eine deutliche Verringerung der Laufzeit zu erkennen.

Tabelle 5.1: Optimierung Probleme [8]

Bereich	Variablen	Ziel Optimierung
Produktion	Auslastung Maschinen	Minimale Kosten, Maximale Effizienz
Elektronik	Größe der Geräte	Minimieren Leistungsverbrauch
Fluggesellschaft	versch. Gesellschaften und Routen	Maximaler Profit
Informatik	Laufzeit	Minimale Laufzeit
Portfolio	Investition in Aktien und Anleihen	Maximale Rendite

Im Bereich der Produktion hatte die Optimierung schon immer eine wichtige Rolle. Das Ziel ist das Beschleunigen des Produktionsablaufes, das Erreichen höchstmöglicher Auslastung der Maschinen und mit der vorhandenen Anzahl an Arbeitskräften das beste Ergebnis zu erreichen. Zudem ist die Auslastung der Maschinen ein wichtiger Faktor für die Effizienz, die zudem das Maximum haben sollte. [7]

5.1 Arten von Optimierungsansätzen

Für die Optimierung gibt es verschiedene Ansätze, um das beste Ergebnis zu erzielen. Das Ziel der Optimierung ist es unter anderem das Auswählen des richtigen Ansatzes. Für jedes Problem gibt es einen Optimierungsansatz. Jede Systemkonfiguration ist eine Lösung (*solution*

S) . Die Menge aller möglichen Systemkonfigurationen ist der Suchraum X . Die *fitness function* f bewertet die Qualität der Lösungen in X . Die *fitness landscape* stellt die Abbildung zwischen Lösungen im Suchraum X und deren entsprechenden Fitnesswerten. Dieser kann statisch, zeitabhängig und selbstreferenziell. Ein statisches Landscape ist fest und verändert sich nicht im Laufe der Zeit. Der Gegenteil dazu sind zeitabhängige Landscapes, die sich mit der Zeit verändern. Selbstreferenzielle Landscapes verändern sich in Abhängigkeit vom Verhalten der Agenten.

Die Auswahl der Optimierungsmethode hängt immer von dem spezifischen Problem ab, da ein Algorithmus für jedes Problem einen eigenen Beigeschmack hat. Das Ziel ist das Finden des globalen Maximums. Das ist in der *Abbildung 4.1* deutlich zu entnehmen. Zudem besitzt jedes Optimierungsproblem Eigenschaften, welche entweder hart oder unkompliziert sind. Einfache Optimierungsprobleme sind:

- Geringe Anzahl an Entscheidungsvariablen
- Differenzierbar
- Ausgewähltes Ziel ist einfach zu berechnen
- Keine oder einfache Nebenbedingungen
- Einfache Bestimmung der Realisierbarkeit
- Ein Ziel
- Deterministisch

Andererseits gibt es Eigenschaften von Optimierungsproblemen, die diese erschweren. Diese lauten wie folgt:

- Hohe Anzahl an Entscheidungsvariablen
- Diskontinuierlich und kombinatorisch
- Ausgewähltes Ziel ist aufwändig zu berechnen
- Einige Nebenbedingungen
- Durchführbarkeit schwer zu bestimmen
- Multimodal/multiple Ziele
- Stochastisch

In den nächsten Abschnitten werden die Arten der Optimierungsansätze näher erläutert und deren Anwendungsbeispiele genannt. [5]

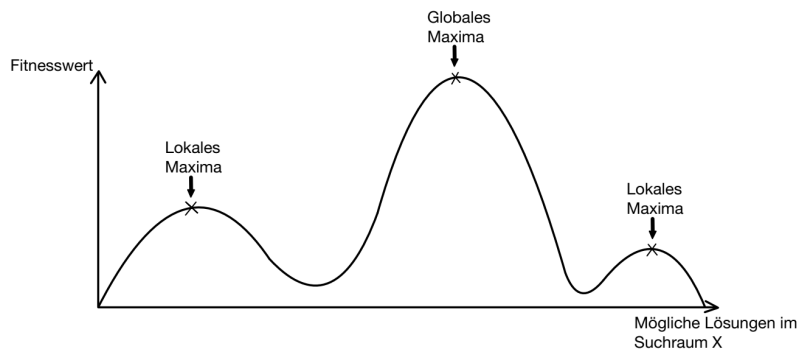


Abbildung 5.1: Wir haben in der Kurve drei Maxima. Das Ziel der Optimierung ist das Ermitteln des globalen Maxima. Dieser erzielt den höchsten Wert der Funktion f und ist somit die beste Systemkonfiguration

5.2 Stochastische Ansätze

Ein Optimierungsansatz ist der Stochastische. Dieser teilt sich auf in trajektoren-basierte (Flugbahn/Kurven-Optimierung) Optimierungsalgorithmen und in populations-basierte Optimierungsalgorithmen. Beispiele für trajektoren-basierte Optimierungsalgorithmen sind: [5]

- Iterated Local Search
- Simulated Annealing
- Stochastic Hill Climbing
- Stochastic Gradient Descent
- Tabu Search
- Greedy Randomized Adaptive Search Procedure

Beispiele für populationsbasierte Optimierungsalgorithmen sind:

- Genetic Algorithms
- Evolutionary Algorithms
- Differential Evolution
- Particle Swarm Optimisation
- Ant Colony Optimisation

Stochastische Optimierungsalgorithmen können im Allgemeinen beschrieben werden. Zuerst wird eine Konfiguration erstellt, welche zu Beginn initialisiert wird. Im Folgenden werden drei Schritte so lange wiederholt, bis ein Abbruchkriterium erfüllt ist. Zuerst wird die Nachbarschaft durchlaufen und es wird ein neuer Nachbar c als Kandidat ausgewählt. Im nächsten Schritt werden einige Kriterien ausgewertet, anhand der *fitness function* f . Im dritten Schritt

wird c_0 mit c initialisiert, wenn $f(c) > f(c_0)$ ist, wobei c_0 der aktuell beste Kandidat ist. Einige Beispiele für Abbruchkriterien sind statisch festgelegte Variablen, die die Anzahl der Iterationen festlegen. Das Feststellen von Verbesserungen ist nicht mehr möglich oder wenn der Algorithmus eine ausreichend hohe Lösung erzielt hat.

Komponenten

Für das Bilden eines solchen Prozesses werden vier Komponenten benötigt. Die Erste ist eine Methode, die für das Generieren der Erstkonfiguration zuständig ist. Als Nächstes eine Funktion für das Suchen und Auswählen eines Nachbarn als nächsten Kandidaten. Die nächste Komponente ist die *cost function* und *fitness function*. Die letzte Komponente ist das Festlegen des Abbruchkriteriums.

5.2.1 Simulated Annealing

Simulated Annealing (SA) ist eine probabilistische Suchtechnik, die physikalische Prozesse nachbildet. Weitere Namen für SA sind Monte Carlo Annealing, Statistical Cooling und Probabilistic Hill Climbing.

Der Algorithmus des SA startet mit der Initialisierung, anhand der zufälligen Anfangsplatzierung. Im nächsten Schritt findet die Bewegung statt. Danach kommt es zur Berechnung der Punktzahl, anhand der durchgeführten Bewegung. Darauffolgend kommt es zur Selektion. Anhand der erreichten Punktzahl wird entschieden, ob die durchgeführte Bewegung angenommen oder verworfen wird. Im letzten Schritt geht es um die Aktualisierung. Ist das gewünschte Ziel erreicht, so wird der Prozess beendet. Falls dies nicht der Fall ist, springt der Algorithmus zum zweiten Schritt - der Bewegung.

SA garantiert Konvergenz, jedoch nur dann, wenn eine ausreichend große Anzahl an Iterationen vorhanden ist. Zudem ist die Konfiguration der Parameter ein sehr wichtiger Aspekt für den Erfolg des Algorithmus. Jedoch scheitert die Methode, wenn die Fitnesslandschaft stark multimodal ist. [5]

5.2.2 Tabu Search

Tabu Search (TS) ist sehr ähnlich zu SA, jedoch wird in TS eine deterministische Akzeptanz/Ablehnungskriterium verwendet. Zu Beginn wird eine Startkonfiguration initialisiert mit der der Algorithmus startet. Ausgehend von der Startkonfiguration wird eine Nachbarschaft von Lösungen erzeugt. Die Nachbarschaftsfunktion $N(x)$ erzeugt eine Menge von benachbarten Lösungen zu x , welche mindestens ein Element enthalten sollte. Daraufhin findet die Auswahl einer neuen Lösung statt. Die Lösung wird als Lösung angewandt, wenn diese die beste Lösung ist und nicht als Tabu Kriterium klassifiziert worden ist. Nach Ausführung des Schrittes wird die Tabu Liste aktualisiert. [5][10]

5.3 Natur-inspirierte Optimierungsheuristiken

Diese Art von Optimierungsansätzen wurden in den letzten Jahren immer gefragter und finden global immer mehr Anwendung in vielen Bereichen. Die verschiedenen Arten von Algorithmen der Heuristik sind beachtlich für die Optimierung und für das Lösen von Problemen in der gesamten Welt. Sei es im Bereich des Ingenieurwesens, Mathematik, Data Science und Bildverarbeitung. [9]

Die natur-inspirierten Optimierungsheuristiken teilen sich in vier Klassen auf. Diese sind physikalische Prozesse, evolutionäre/genetische Algorithmen, memetische Algorithmen und Schwarmverhalten.

5.3.1 Evolutionäre Algorithmen

Evolutionäre Algorithmen (EA) sind metaheuristische Optimierungsverfahren, die von der Evolution natürlicher Lebewesen inspiriert ist. In Anlehnung an die Natur werden Lösungskandidaten für ein bestimmtes Problem künstlich evolviert, EA sind also naturanaloge Optimierungsverfahren. EAs führen in den meisten Fällen nicht zur besten Lösung, jedoch ist das erreichte Ziel meist ausreichend. Der Ablauf eines evolutionären Algorithmuses ist in *Abbildung 4.2* zu sehen. [5]

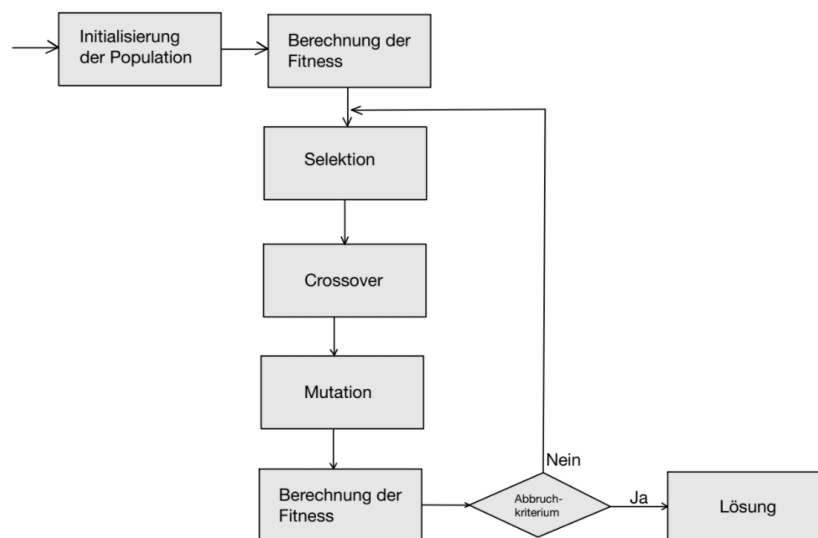


Abbildung 5.2: Chronologische Reihenfolge eines evolutionären Algorithmuses

5.4 Metaheuristiken

Eine weiterer Optimierungsansatz ist die Anwendung von Metaheuristiken. Metaheuristiken sind Algorithmen in der Informatik, die verwendet werden, um eine näherungsweise Lösung zu Optimierungsproblemen zu finden und sind im Allgemeinen nicht deterministisch. Zudem sind Methoden von Metaheuristiken nicht problemspezifisch. Liegt beispielsweise ein Problem vor, das nicht effizient gelöst werden kann, wenn es sich um ein komplexes kombinatorisches Optimierungsproblem handelt. So werden für die Lösung solcher Probleme Metaheuristiken für angewendet.

Metaheuristiken werden in metaphorisch basierte und nicht-metaphorisch basierte Algorithmen unterteilt. Die verbreitetsten metaphorisch basierten Methodiken sind Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Water Waves Optimization (WWO), Clonal Selection Algorithm (CLONALG), Chemical Reaction Optimization (CRO), Harmony Search (HS), Sine Cosine Algorithm (SCA), Simulated Annealing (SA), Teaching-Learning-Based Optimization (TLBO) und League Championship Algorithm (LCA).

Nicht-metaphorisch basierte Metaheuristiken sind unter anderem Tabu Search (TS) und Variable Neighborhood Search (VNS). Im Abschnitt 5.6 wird genauer auf die Auswahl der Optimierungsmethode in dem vorliegenden Problem eingegangen. Dabei wird ersichtlich, weshalb die Anwendung von Metaheuristiken die beste Wahl ist. Denn hierbei handelt es sich ebenfalls um Kombinationsproblem und der Suchraum für die Auswahl der besten Kombination ist zu groß. Aufgrund des großen Suchraumes ist auch die Auswahl eines effizienten Lösungsalgorithmus schwer, da für jede Kombination der Arbeitsvorgänge die Performance berechnet und verglichen werden muss. [14]

Nach dem Lesen der oberen Abschnitte wird deutlich, dass Methodiken wie zum Beispiel GA, PSO oder SA als Metaheuristiken erfasst werden. Jedoch fallen diese auch unter die Gattung der stochastischen Optimierungsansätze. Demnach sind Metaheuristiken ähnlich zu stochastischen Ansätzen. Um auftretende Fragen zu klären wurde die *Abbildung 5.3* erstellt, die die Verteilung deutlich darstellt. Zu erkennen sind Optimierungsansätze und Methodiken, die in den vorherigen Abschnitten genannt worden sind. Demnach zählen einige Methodiken als Metaheuristiken und als stochastische Optimierungsansätze.

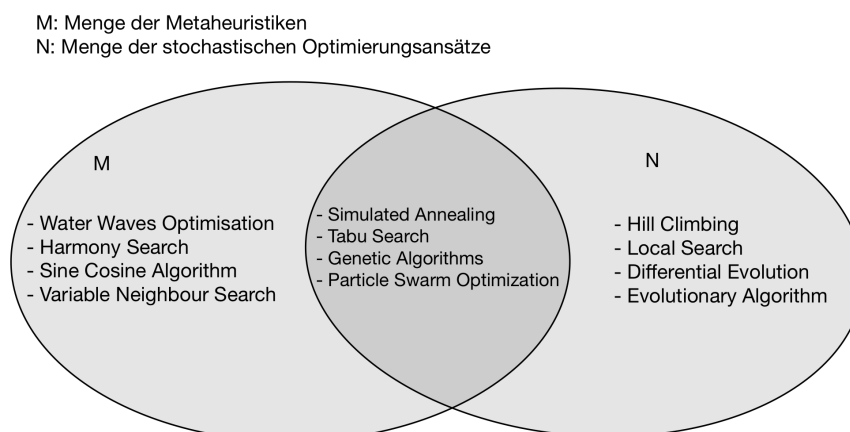


Abbildung 5.3: Schnitt der Menge der Metaheuristiken mit der Menge der stochastischen Ansätze

5.5 Unification Ansätze

Die Vereinigung der Strukturen in den natur-inspirierenden Optimierungsheuristiken weist einige Vorteile auf. Das Vergleichen verschiedener Optimierungsheuristiken ist wesentlich einfach. Dies würde zu einer Erleichterung des Verstehens von neuartigen Ansätzen beitragen. Des Weiteren bietet sich die Möglichkeit für den Austausch von Operatoren. Aus diesem Grund ist eine Hybridisierung möglich und das Kombinieren von Vorteilen von verschiedenen Algorithmen ist möglich. Der letzte Aspekt ist die Vereinfachung der Implementierung, da Algorithmen aus wiederverwendbaren Komponenten aufgebaut werden können. [5]

5.6 Auswahl Optimierungsansatz

In diesem Abschnitt handelt es sich um die Auswahl des Optimierungsansatzes. Es gibt viele Ansätze. Jedoch weisen einige Problematiken auf. Denn in solch einem Problem, wie in der Optimierung eines realistischen Produktionsablaufes, ist es nicht möglich eine perfekte Lösung zu finden. Die Verwendung von mathematischen Ansätzen ist in solch einem Problem nicht möglich. Diese berechnen die perfekte Lösung für ein Optimierungsproblem. Da es sich hierbei auch um ein kombinatorisches Problem handelt, ist die Anwendung von Metaheuristiken ein guter Ansatz, um eine zufriedenstellende oder hinreichend gute Lösung zu finden. Zudem ist der Rechenaufwand für die Entwicklung einer Lösung deutlich geringer als iterative Methoden.

Für die Auswahl von mathematischen Optimierungsansätzen hätten wir ein Problem. Das zu simulierende Modell besitzt viele verschiedene Möglichkeiten, um den Produktionsfluss zu generieren. Für die Anzahl der verschiedenen Kombinationen wird die Menge der Kombinationen aus allen möglichen Abfertigungsschritten berechnet. Diese ist ebenfalls ein Maß für die Komplexität eines Modells.

Die Komplexität eines Problems ist der Ressourcenverbrauch eines Algorithmus zur Lösung eines Problems. Die Ressourcen beschreiben die Anzahl der benötigten Rechenschritte oder den Speicherbedarf.

Angenommen wir haben neun Abteilungen in dem Unternehmen. So erhält man für die Anzahl der Möglichkeiten $9!$. Dies ist wiederum eine enorm hohe Anzahl an Möglichkeiten und Speicherbedarf, dieser ist nach der obigen Definition äquivalent zum Ressourcenverbrauch. Teilt man die Aufträge spezifisch auf die Maschinen auf, so steigt die Anzahl der Möglichkeiten und den Speicherbedarf. Dies ist wiederum eine Steigerung der Komplexität des Modells. [18]

Demnach ist die Komplexität des Modells äquivalent zu der Anzahl der Möglichkeiten des Suchraumes oder der verschiedenen Möglichkeiten den Produktionsablauf darzustellen. In dem zu simulierendem Modell haben wir verschiedene Möglichkeiten das Produkt abzufertigen. Jedoch wird die Anzahl der Möglichkeiten ein wenig reduziert. Dies liegt an der durchschnittlichen Priorität der Abfertigung. Die größte Anzahl an Rohmaterialien muss meist als ersten Schritt in die Abteilung der Lasermaschinen, um diese in die gewünschte Form zu schneiden. Dies ist meist der erste Arbeitsvorgang. Schließlich ist in der Produktion die Lackierung oder das Pulvern meist der letzte Arbeitsvorgang, bevor die Materialien in den LKW geladen und ausgeliefert werden. Wie erhalten wir die Kombination der möglichen Arbeitsschritte, um

die Anzahl der Möglichkeiten zu berechnen und somit die Komplexität unseres Modells zu berechnen. Wir haben in diesem System verschiedene Abteilungen:

- Laser
- Abkant
- CNC
- Rohrlaser
- Stanzerei
- Schweiserei
- Qualitätssicherung (QS)
- Pulverei
- KTL Beschichter

Um die Anzahl der verschiedenen Möglichkeiten zu berechnen wird die Anzahl der Permutationen verwendet, da jede Abteilung nur einmal in den Arbeitsvorgängen nur einmal vorkommt. Somit erhalten wir für die Anzahl der verschiedenen Kombinationen, wobei n die Anzahl der verschiedenen Arbeitsschritte ist:

$$\sigma = n! = 9! = 362.880$$

Somit erhalten wir für den mathematischen Ansatz das Problem, dass wir alle 362.880 Möglichkeiten berechnen und testen müssen, ob eine Optimierung erfolgt oder nicht. Durch die Reduzierung der Arbeitsschritte um $n-1$, da ein Großteil der Rohstoffe in der Lasermaschine zugeschnitten werden müssen, erhalten wir $n! = 8! = 40.320$ Möglichkeiten.

Für das Lösen dieses Problems wird eine sehr hohe Laufzeit und ein sehr hoher Berechnungsaufwand benötigt. Aufgrund dessen ist es effizienter, wenn für die Lösung dieses Problems Metaheuristiken verwendet werden, um ein ausreichend zufriedenstellendes Ergebnis zu berechnen.

6 Simulation

Nach der Einführung in die Grundlagen und die Arten der Optimierungsansätze, handelt es sich in diesem Kapitel um die Simulation. Dabei werden die Nutzen und Vorteile aufgrund der Anwendung einer Simulation aufgelistet. Im darauffolgenden Abschnitt geht es um die Simulation eines realistischen Produktionsablaufes. Dabei wird kurz auf das Unternehmen eingegangen. Im Weiteren werden Parameter ausgewählt, die für die Produktion ausschlaggebend sind und für das Unternehmen eine große Rolle spielen. Nachfolgend werden auf die Ansätze des Unternehmens eingegangen, die geplant sind, um die Effizienz und den Produktionsfluss zu steigern. Anschließend wird die erstellte Simulation erklärt und das Vorgehen für die Erstellung der Simulation näher erläutert.

6.1 Vor- und Nachteile einer Simulation

Der vorliegende Abschnitt beinhaltet die Vorteile und die Nachteile, die durch die Anwendung von Simulationen entstehen. Vorteile für die Verwendung einer Simulation in einem System sind:

- Risikofreie Umgebung zum Testen
- Sichern von Geld und Zeit
- Effektive Umsetzung von Ideen
- Einsicht in komplexe Systemdynamiken
- Verringerung von Ineffizienz
- Hilfe bei Unsicherheiten/Minimierung von Risiken
- Ausgabe von wichtigen Variablen
- Visualisierung (2D/3D)
- Erhöhte Genauigkeit
- Verbesserung Planungsprozesse

Die Darstellung von Szenarien anhand einer Simulation reduziert das Risiko von möglicherweise auftretenden Problemen. Anhand dieser ist es möglich Ideen zu testen, ohne diese in Wirklichkeit realisieren. Denn anhand dieser Tests ist es dem Unternehmen möglich im Vorhinein zu erkennen, ob Umsetzungen im Unternehmen möglich sind. Dies ist ein wichtiger Gesichtspunkt. Angenommen die Simulation wäre kein Werkzeug, um Ideen zu testen. Dies würde bedeuten, dass das Unternehmen ihre Ideen in der realen Welt testen müsste. Sollte es beim Testen in der realen Welt zu Konflikten kommen, kosten diese dem Unternehmen sehr

viel Geld und zudem führen solche Umsetzungen meist dazu, dass die Produktion angehalten werden müsste und es zu einem Produktionsstop kommen würde. Aufgrund von diesem Aspekt verschafft sich das Unternehmen den Vorteil, dass sie durch das Senken des Risikos Geld und Zeit sparen.

Die Hilfe bei Unsicherheiten ist ein weiterer nennenswerter Punkt. Ein gutes Beispiel dient das hier zu simulierende Unternehmen. Dieses Unternehmen hat den Bau eines Zentrallagers in Planung. Das Bauen eines Zentrallagers ist eine kostenintensive Investition. Deshalb muss die Planung sehr gut kalkuliert werden, ob sich diese Investition lohnt. Anhand der Simulation ist es möglich die Idee in der Simulation zu testen und mithilfe der erzielten Ergebnisse zu entscheiden, ob diese Investition gemacht werden soll. Die Unsicherheit des Unternehmens ist abnehmbar Dank der Resultate der Simulation.

Ein weiterer Vorteil der Simulation ist die Ausgabe von wichtigen Variablen. Für die Produktion des Unternehmens, welches in dieser Arbeit simuliert wird, sind unter anderem die Auslastung der Maschinen, Anzahl produzierter Teile, Zeit, die in der jeweiligen Abteilung benötigt wird für die Fertigung des Produktes.

Die Simulationssoftware Anylogic bietet die Simulation in der zwei- und dreidimensionalen Ebene an. Wird ein Modell in einer dreidimensionalen Ebene simuliert, so fällt das Verstehen von Konzepten und Ideen deutlich einfacher. Zudem ist es möglich diese Visualisierung den Führungspositionen vorzulegen und Erläuterungen der Herangehensweise zu vereinfachen. [11]

Anschließend ist der Support von AnyLogic zu erwähnen. Dieser bietet dem Nutzer eine schnelle und hilfreiche Unterstützung bei auftretenden Fragen, während der Erstellung der Software. So erhält man zum Beispiel meist binnen wenigen Werktagen eine Antwort.

Trotz all dieser Gründe dürfen die dadurch entstehenden Nachteile nicht übersehen werden. Der erste Nachteil entsteht aufgrund vom fehlerhaften Programmieren und Erstellen der Simulation. Da die Erfahrung beim Verknüpfen von Simulation mit einer Programmiersprache fehlt, so kommt es trotz Programmierkenntnisse oft zu Fehlern.

Für die Simulation eines realistischen Modells müssen alle Methoden realistisch und korrekt gewählt und programmiert werden. Hinzu kommt, dass das Erzielen von nennenswerten Ergebnissen eine gewisse Zeit benötigt. Bis eine zufriedenstellende Simulation erstellt wurde, vergingen einige Wochen.

Anschließend ist das Verstehen des Systems, welches simuliert werden soll, ein großes Problem für die Erstellung. Die Entwicklung der Simulation für den Produktionsablauf innerhalb des Unternehmens muss man das System, das für die Produktion gewählt wurde, verstehen. Das Beginnen der Programmierung von der Simulation, ohne das System und alle beteiligten Faktoren zu verstehen, würde zu Unklarheiten führen. Spätestens bei der Vorschau der Simulation vor dem Geschäftsführer des Unternehmens würden sich diese erkennbar machen. [11]

6.2 Simulation eines realistischen Produktionsablaufes eines Unternehmens

In diesem Passus wird auf den realistischen Produktionsablauf eingegangen. Für diese fokussieren wir uns auf ein mittelständisches Unternehmen. Die Simulation wird hierbei verwendet, um den Produktionsfluss zu verbessern und die Effizienz zu steigern. Zudem ist für dieses Unternehmen die Auslastung der Maschinen ein sehr wichtiger Fokus. Denn es handelt sich um mehrere große Laserschneid-, Rohrlaser-, CNC-Maschinen, Abkantpressen und Abteilungen, wie eine Schweißerei und Pulveranlage. Diese sollten idealerweise zu 100 Prozent ausgelastet sein. Die Darstellung der Auslastung wird mithilfe der Statistiken wie in *Abbildung 6.1* visualisiert. Auf der x-Achse sind die verschiedenen Ressource Pools. Die y-Achse (Float Werte von 0.0 bis 1.0) spiegelt die Auslastung der Maschinen von Null bis Eins, wobei die Null für eine Auslastung von Null Prozent und die Eins für eine Auslastung von 100 Prozent steht.

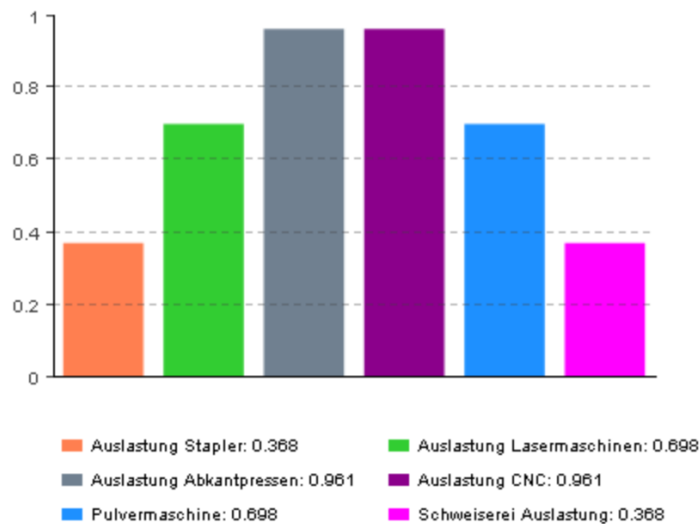


Abbildung 6.1: Die Simulationssoftware Anylogic bietet die Möglichkeit Ressource Pools zu erstellen. Das sind wie Gruppen von Maschinen. Zum Beispiel haben wir ein Ressource Pool für die Lasermaschinen, Abkantpressen, CNC Maschinen oder Schweißerei. Daraufhin können Statistiken erstellt werden, die die Auslastung (Utilization) der jeweiligen Ressource Pools ausgeben.

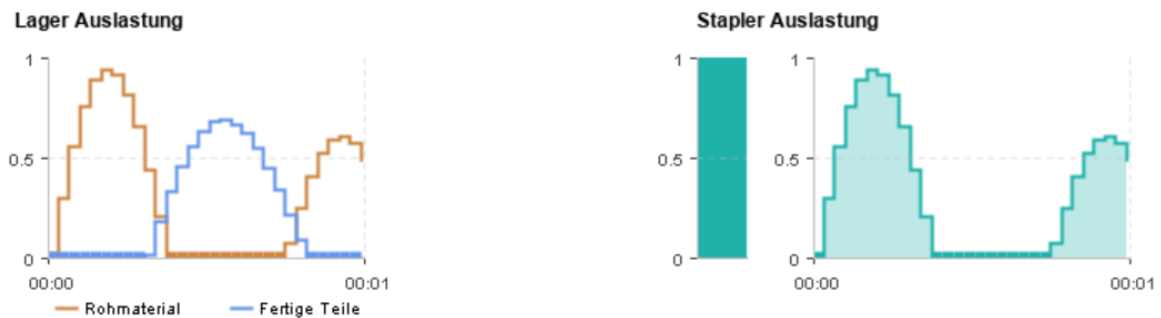


Abbildung 6.2: Die Ausgabe von Auslastung von dem Lager und dem Stapler in Form von Statistiken. Zudem ist die prozentuale Auslastung zwischen Rohmaterialien und fertigen Teilen in dem Lager darstellbar. Da es sich in der Produktion größtenteils um schwere Materialien handelt, ist ein Stapler, mit dem Lastkraftwagen, eines der wichtigsten Fahrzeuge. Dieser sollte aufgrund seiner begrenzten Stückzahl so effizient wie möglich genutzt werden.

Anschließend kann die Anzahl der produzierten Werkstücke und die Zeit, die die Maschinen für den jeweiligen Auftrag benötigen, ausgegeben werden. Das Unternehmen will in naher Zukunft ein Zentrallager bauen. Diese kostenintensive Investition wird ebenfalls simuliert und getestet, ob die Investition den Produktionsfluss und die Effizienz steigert.

Zudem sind weitere Aspekte vorhanden, die erwähnt werden müssen, um den Realismus der Simulation zu gewährleisten. Denn das Hauptziel einer Simulation ist die Steigerung der Effizienz.

Demnach ist der erste Gesichtspunkt die Beachtung, ob es sich bei der Produktion um eine Serienfertigung oder um die Fertigung eines Prototypen handelt. Die Bearbeitung von Serienteilen ist deutlich einfach und es müssen wenige Schritte durchgeführt werden, um die Produktion zu starten und eine hohe Stückzahl an Produkten zu produzieren. Im Fall einer Serienfertigung müssen die Maschinen und Vorrichtungen für die Fertigung in den jeweiligen Abteilungen jeweils einmal eingestellt werden und es ist möglich eine hohe Anzahl an Produkten auf einmal zu produzieren ohne die Maschine umzustellen. Nach der Besprechung mit dem Geschäftsführer wurde mitgeteilt, dass im Durchschnitt 15 Minuten für die Preparation der Maschinen benötigt werden, damit die Produktion gestartet werden kann. Zudem stellt sich nebensächlich die Frage, ab welcher Anzahl es sich lohnt die Produktion der Teile zu starten.

Die Produktion von Prototypen ist wesentlich zeit- und kostenintensiver und lohnt sich in der Simulation nicht, denn in den meisten Fällen handelt es sich um Einzelteile. Das Produzieren von Prototypen ist meist ein einziger Produktionsablauf, in dem die Maschinen und Vorrichtungen einmal eingestellt werden und dann in den meisten Fällen nicht mehr verwendet werden. In der erstellten Simulation handelt es sich um die Fertigung von Serienteilen mit einer höheren Anzahl von Teilen, denn anhand dieser ist ein Produktionsfluss gegeben. Mittels diesen Flusses ist es möglich eine Simulation des Systems zu erstellen und diese gegebenenfalls zu optimieren.

6.2.1 Randdaten

Das zu simulierende System ist ein Metallbau¹ Unternehmen in der Marktgemeinde Burtenbach. Die Firma hat ungefähr 120 Angestellte und produziert jegliche Arten von Metallkonstrukten. Anhand mehrerer Abteilungen mit verschiedenen Arten von Maschinen ist es in diesem Unternehmen möglich komplexe Bauteile und Baugruppen zu fertigen. Die verschiedenen Abteilungen sind aus dem Abschnitt 6.1 zu entnehmen. Das Ziel des Unternehmens ist die Verbesserung der Effizienz und die Steigerung der Auslastung der Maschinen und Abteilungen, um die Anzahl der gefertigten Produkte zu erhöhen.

6.3 Bedeute Parameter für die Produktion

Dieser Abschnitt beschreibt die maßgeblichen Parameter, die für das Unternehmen von großer Bedeutung sind. Diese lauten wie folgt:

- Auslastung der Maschinen
- Anzahl Produzierter Teile/Ausbringungsmenge
- Zeit, die in der jeweiligen Abteilung benötigt wird
- Rüstzeit/Umstellungszeit
- Produktionszeit
- Produktionsdurchlaufzeit
- Betriebsstättenplanung/Fabrikplanung
- Ergonomische Arbeitsweise
- Kanban²

Für die Rüstzeit der Maschinen wird ein Durchschnitt verwendet, den das Unternehmen aus den langjährigen Erfahrungen ermittelt hat. Demnach beträgt die Rüstzeit im Durchschnitt ungefähr zwischen 15 und 20 Minuten. Aufgrund dessen wird eine Konstante angelegt, die mit der durchschnittlichen Rüstzeit initialisiert wird. Der Betrieb der Maschinen benötigt Energie und haben zudem enorme Anschaffungskosten. Für die Deckung dieser Kosten sollten die Maschinen idealerweise zu 100 Prozent ausgelastet sein und Produkte zu produzieren, um Gewinn erzielen zu können. Zudem sollten ebenso weitere Abteilungen und vor allem Mitarbeiter eine hohe Auslastung besitzen. Die Produktionszeit ist die tatsächliche Produktionszeit. Der nächste Parameter ist die Produktionsdurchlaufzeit. Diese beschreibt die Zeitspanne zwischen dem Eintreffen eines Fertigungsauftrags am Ersten zu durchlaufenden Arbeitsplatz und der Bereitstellung des abgeschlossenen Auftrags im Fertigwarenlager oder Versand. [15]

¹<https://stern-metallbau.com/>

²Kanban ist eine Methode in der Produktionssteuerung nach dem Zug-Prinzip, die permanente Eingriffe einer zentralen Steuerung in den Produktionsablauf überflüssig macht und sich ausschließlich am Kundenbedarf orientiert.[21]

Die Betriebsstättenplanung/Fabrikplanung ist die Neuplanung oder Änderung eines Fabrikbetriebs. Diese werden bei Neugründung einer Fabrik oder Änderung eines bestehenden Betriebes erstellt. Dabei zählt die Architektur, Gebäudeplanung, Produktionsanlagenplanung, Anlagenlayoutplanung, Personalplanung als auch die Aufgaben der Fabrikorganisation. [16]

Weitere wichtige Parameter wie die ergonomische Arbeitsweise oder die Flexibilität sind von enormer Wichtigkeit. Jedoch besteht keine Möglichkeit diese in der Simulation einzubauen und grafisch darzustellen oder statistisch auszugeben, da diese Parameter nicht messbar sind.

6.4 Ansätze des Unternehmens für die Optimierung des Produktionsablaufes/-flusses

Für die Optimierung des Produktionsablaufes hat das Unternehmen ebenfalls einen Vorschlag. Diese wollen demnächst ein Zentrallager bauen. Das Ziel des Baus ist die Verbesserung des Produktionsflusses. Der Bau eines solchen Projekts ist sehr kostenspielig. Demnach ist das Testen und Simulieren des Ansatzes ein wichtiger Ansatzpunkt, um Urteilen zu können, ob sich eine solche Investition lohnt.

6.5 Verfahrensweise

In dem folgenden Abschnitt handelt es um die Verfahrensweise für die Erstellung der Simulation. Zu Beginn der Arbeit wurde festgelegt, welche Punkte für die Simulation benötigt werden und welche Themen dafür behandelt werden müssen. Nach dem Treffen mit dem Professor wurden wichtige Punkte, Aspekte und Ziele festgelegt. Basierend auf diesen Punkten wurde begonnen, die Simulation zu erstellen. Beginnend mit dem Lernen anhand der Tutorials und dem Erlernen der Funktionsweisen von verschiedenen Blöcken. Im nächsten Schritt wurde mit dem Erstellen der eigentlichen Simulation begonnen.

6.5.1 Umrisse des Unternehmens in Anylogic übernehmen

Als ersten Schritt wurden die Umrisse des Unternehmens mit den verschiedenen Gebäuden genommen und wurden maßstabsgetreu in die Software übernommen. Danach wurden die Wände der Hallen und das Bürogebäude wurde es als geschlossenes Gebäude erstellt. Für die Produktionshallen wurde kein Dach erstellt, da wir anhand dessen in dem Produktionsablauf sehen und den Ablauf der Produktion verfolgen können. Anschließend wurden dreidimensionale Objekte wie Container oder Maschinen eingefügt.

6.5.2 Einfügen der Maschinen

Nach dem Erstellen des Geländes und der Produktionshallen beginnt das Einsetzen von den Maschinen anhand dem Lageplan des Unternehmens. Die eingesetzten Geräte sind statische Maschinen. Dies ist ein Attribut, welches ebenfalls bei der Erstellung einer Ressource benötigt wird. Es wird zwischen portablen, statischen und beweglichen Ressourcen unterschieden. Statische Ressourcen sind an einem bestimmten Ort gebunden und können sich nicht bewegen oder verschoben werden. Bewegliche Ressourcen können sich unabhängig bewegen. Portable Ressourcen können von Agenten oder durch bewegliche Ressourcen verschoben werden. Ressourcen sind Objekte die Agenten verwenden, um eine bestimmte Aktion durchzuführen. Ein Agent muss sich die Ressource beschaffen, die Aktion durchführen und die Ressource dann wieder freigeben.

Die Agenten, die in der Simulation dargestellt werden, sind die jeweilig dafür ausgewählten dreidimensionalen Objekte. Zudem können Agenten weitere unterschiedliche Dinge darstellen:

- Fahrzeuge
- Maschinen
- Ausrüstungsgegenstände
- Projekte
- Produkte
- Ideen
- Organisationen
- Investitionen
- Grundstücke
- Menschen in verschiedenen Rollen

Agenten sind Hauptbestandteile des Anylogic Modells. Ein Agent ist eine Einheit des Modells, die Verhalten, Speicher (Verlauf, Historie), Timing, Kontakte, etc. haben kann. Innerhalb eines Agenten können Variablen, Parameter, Ereignisse, Zustandsdiagramme, Bestands- und Flussdiagramme definieren. Zudem besteht die Möglichkeit andere Agenten einzubetten und Prozessflussdiagramme hinzuzufügen. Ist die Vollversion der Software verfügbar, so besteht die Möglichkeit so viele Agententypen zu definieren, wie es verschiedene Typen von Agenten benötigt wird. Jedoch ist anzumerken, dass in der vorliegenden Arbeit die Personal Learning Edition für Studenten verwendet wird. Diese beschränkt die Anzahl der Agententypen auf zehn ein. [17]

Das Verstehen der agentenbasierten Simulation wird durch *Abbildung 6.3* veranschaulicht. Jedes Objekt, welches für den Produktionsablauf benötigt wird, ist ein Agent. So sind zum Beispiel Paletten, Stapler, Lasermaschinen und Abkantpressen Agenten. Objekte, die nur für die Visualisierung benötigt werden, können einfach anhand von dreidimensionalen Objekten eingefügt werden, ohne die Erstellung eines neuen Agententypen. Ein Beispiel für ein solches Objekt sind zum Beispiel die blauen Abfallcontainer in *Abbildung 6.3*.

- >  Abkantpresse
- >  CNC
- >  ForkliftTruck
- >  HubwagenFahrer
- >  Lasercutter
- >  **Main**
- >  Pallet
- >  Schweißer
- >  Truck

Abbildung 6.3: Agenten für die Simulation des Produktionsablaufes

Für den Einsatz der Maschinen in der Simulation wird für jede Maschine ein Punktknoten erstellt, welcher für den Standort der Maschinen steht und damit auch verknüpft werden kann. Anhand der großen Auswahl an dreidimensionalen Objekten, die die Software zur Verfügung stellt, ist es möglich einen Großteil der Maschinen, Gebäude, Personen und viele weitere Objekte in den 3D-Objekten zu finden und einzufügen. In *Abbildung 6.4* sind alle Bereiche zu sehen, die die Simulationssoftware bereitstellt.

- ▶ People
- ▶ Buildings
- ▶ Road Transport
- ▶ Rail Transport
- ▶ Maritime Transport
- ▶ Military
- ▶ Warehouses and Container Terminals
- ▶ Health
- ▶ Manufacturing
- ▶ CNC Machines
- ▶ Energy
- ▶ Mining
- ▶ Office
- ▶ Supermarket
- ▶ Airport
- ▶ Miscellaneous
- ▶ Boxes
- ▶ Bottles
- ▶ Roads
- ▶ Tanks
- ▶ Pipes
- ▶ Geometric primitives

Abbildung 6.4: Dreidimensionale Objekte, die Anylogic anbietet, die für die Simulation benötigt werden. Die Software hat Objekte in vielen verschiedenen Bereichen, weshalb es möglich ist für jede Art von Bereich die passenden Objekte zu finden.

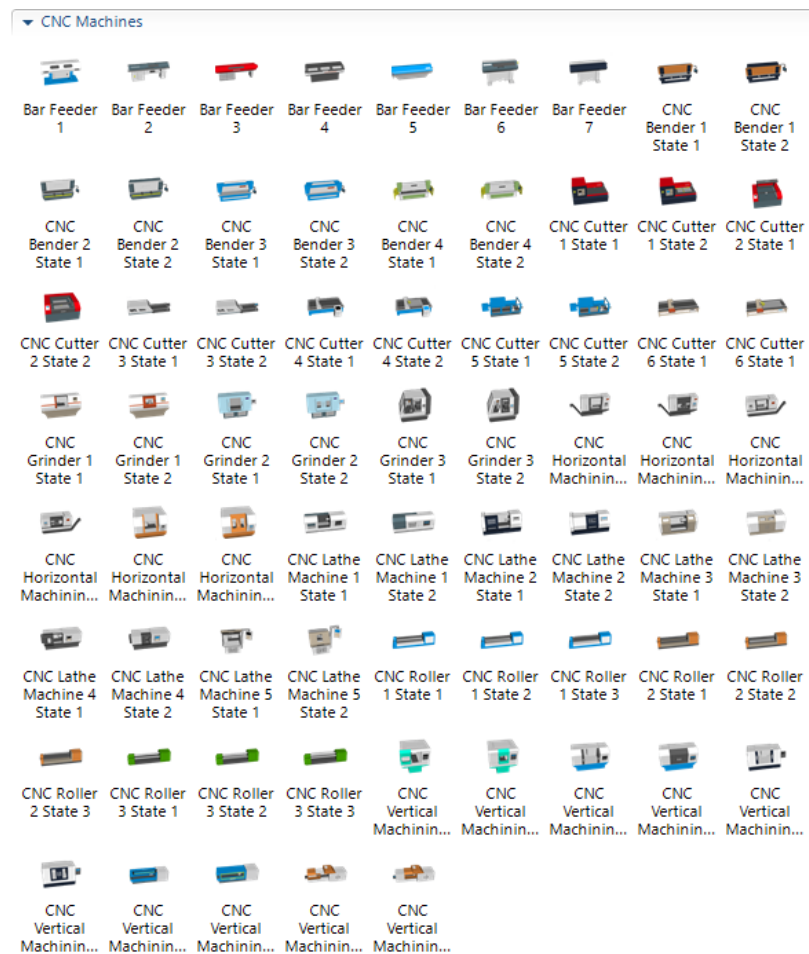


Abbildung 6.5: Dreidimensionale Objekte für CNC Maschinen zu sehen, die für die Erstellung der Simulation benötigt werden

Nach dem Auswählen und Einfügen der statischen Maschinen, wie zum Beispiel die Lasermaschinen, Abkantpressen und CNC-Maschinen wurden bewegliche Agenten eingesetzt. Diese sind unter anderem Fahrzeuge wie Lastkraftwagen oder Stapler. Diese werden in das System integriert und Parameter wie Geschwindigkeit werden in den Attributen festgelegt.

Erstellen eines Netzwerkes (Network)

Ein Netzwerk, zu sehen in *Abbildung 6.6*, ist eine Menge von Knoten, die durch Segmente miteinander verbunden sind. Wenn eine netzwerk- oder layoutbasierte Modellierung benötigt wird, dann wird ein Netzwerk erstellt. Diese finden Verwendung, wenn die modellierten Prozesse in einem bestimmten physischen Raum ablaufen und die Bewegung von Agenten und Ressourcen beinhalten. Diese sind zum Beispiel eine Produktionsstätte oder ein Logistikprozess.

Die Netzwerktopologie wird definiert, indem spezifische Knoten und Pfade zum Beispiel über eine als Hintergrund verwendete Gebäudekarte gezeichnet werden. Ein Knoten in einem Netzwerk definiert den Ort an dem ein Agent aufhalten könnte. Ein Pfad der Knoten miteinander

verbindet, stellt Routen, die die Agenten verwenden, in der Simulation dar. Besteht zwischen den Knoten keine Verbindung anhand von Pfaden so erhalten wir Fehlermeldungen, die das korrekte Durchführen und Ablaufen der Simulation verhindert.

Geschwindigkeiten von Ressourcen und Agenten können individuell gegeben oder durch einen Parameter bestimmt werden. So können unterschiedliche Geschwindigkeiten definiert werden für einen spezifischen Agenten. Ein Beispiel für die Auswahl von unterschiedlichen Geschwindigkeiten ist ein Stapler und ein Lastkraftwagen. Die Geschwindigkeit eines Lastkraftwagens auf dem Firmengelände beträgt zehn Kilometer pro Stunde. Jedoch ist die Geschwindigkeit eines Staplers deutlich geringer. Zudem besteht die Möglichkeit einen Schieberegler einzusetzen, wodurch Geschwindigkeiten während der Simulation dynamisch verändert werden können. Ein Netzwerkelement wird automatisch erstellt, wenn zwei oder mehr Netzwerkelemente (Pfad oder Knoten) miteinander verbunden werden. Zudem können Eigenschaften des Netzwerkes bearbeitet werden, wie zum Beispiel die Sichtbarkeit in der Simulation.

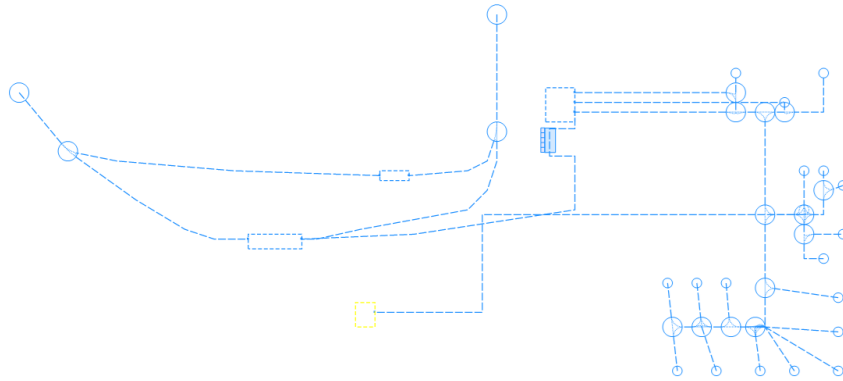


Abbildung 6.6: Netzwerk eines Produktionsablaufes der Simulation

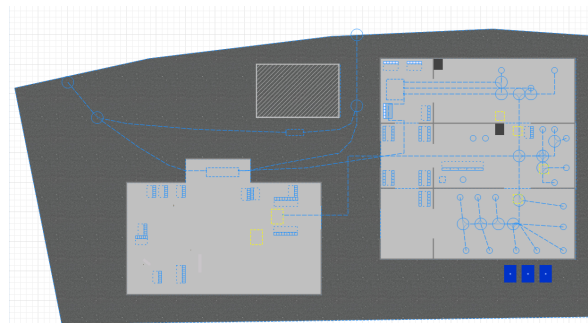


Abbildung 6.7: Eingesetztes Netzwerk in die Simulation

6.5.3 Erstellen von Blöcken (Blocks)

Für die Erklärung von Blöcken und Methoden sind keinerlei Definitionen oder Quellen vorhanden. Aufgrund dessen wird die Logik, Blöcke und Methoden mit eigenen Worten erklärt. In den vorherigen Abschnitten handelte es sich um die Erstellung der Umrisse des Unternehmens, das Einfügen der Maschinen und das Erstellen eines Netzwerkes.

Startet die Simulation so ist nur das Unternehmen mit den Objekten, Gebäuden und ein Netzwerk zu sehen. Jedoch findet keine Simulation statt. Das liegt daran, dass wir Methoden und Blöcke oder eine Logik einfügen müssen, um den Objekten, Ressourcen und Agenten zu sagen, was sie zu tun haben. Die vorhandene Bibliothek von Anylogic beinhaltet Knoten und Pfade, die für die Erstellung von Netzwerken benötigt werden. Des Weiteren sind Blöcke enthalten, die es ermöglichen die Simulation zu erstellen, zu steuern und die Logik hinter der Simulation anzulegen und anzufertigen. Für das bessere Verständnis dient die *Abbildung 6.8*. Für die Erstellung einer Simulation des Produktionsablaufes werden Ressource Pools von Agenten erstellt. Diese sind zum Beispiel Stapler. Diese sind anhand der RackPick, RackStore oder MoveTo Methoden in der Lage Palleten (Agenten) von einem Standort zum nächsten Standort zu bewegen und somit den Transport an verschiedene Arbeitsschritte zu simulieren. Ein Source Block, wie in der *Abbildung 6.8* zum Beispiel *sourceDeliveryTrucks* oder *sourceShippingTrucks*, generiert Agenten und zählen meistens als Startsequenz des Prozessmodells. Es können Standardagenten oder benutzerdefinierte Agenten dafür erstellt werden. Danach können weitere Einstellungen definiert werden, wie der Zeitpunkt und die Anzahl der Agenten, die eingeführt werden sollen.

Um die Dauer von Produktionsschritten zu simulieren werden Delays in den jeweiligen Arbeitsschritten eingefügt. Diese können mithilfe von Parametern festgelegt und verändert werden. Um die Auslastung der Maschinen und Abteilungen auszugeben, wird die Seize Methode verwendet. Diese ergreift den jeweiligen Agenten oder Ressource und hält diese so lange bis die Release Methode aufgerufen wird. Nach der Seize Methode wird ein Delay eingesetzt, um die Dauer des jeweiligen Produktionsschrittes zu übergeben.

Möchte man zum Beispiel den Produktionsschritt des Laserschneides simulieren, so werden mindestens die Seize, Delay und Release Methode benötigt. In der Seize Methode wird die Palette von der Lasermaschine ergriffen und verarbeitet. In der Delay Methode wird die Dauer des Produktionsschrittes definiert. Nachdem die Dauer des Produktionsschrittes erfüllt ist, muss die Release Methode aufgerufen werden, um die Palleten von der Lasermaschine freizugeben und zum nächsten Arbeitsschritt zu bringen.

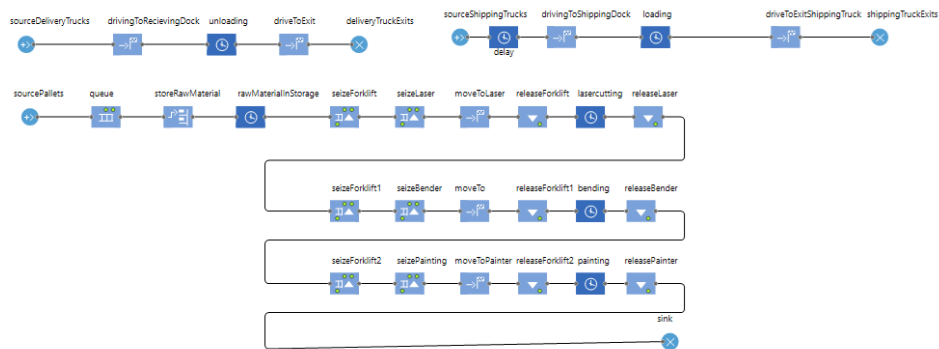


Abbildung 6.8: Eingesetztes Netzwerk in die Simulation

Ein Problem, welches während der Erstellung der Simulation entstand, war der Transport der Paletten. Aufgrund der im kleinen Maßstab gehaltenen Tutorials musste selbst getestet werden, wie die Paletten von dem Stapler erfasst und zu den jeweiligen Arbeitsvorgängen gebracht werden. So wurde am Anfang der Erstellung der Simulation die MoveTo Methode verwendet. Diese erfüllte den Zweck, dass die Paletten an den jeweiligen Standort gebracht wurden, jedoch ohne den Stapler. So wurden die Paletten von alleine in die jeweilige Abteilung gebracht. Ein Ansatz für die Behebung des Problems war die Verwendung der RackPick Methode, welche in den Tutorials verwendet wurde. Das hat nur für einen Arbeitsschritt geklappt. Denn so wurden Paletten von dem Regal geholt und an die Laser Abteilung gebracht. Da in der Simulation ein Produktionsablauf mit mehreren Arbeitsschritten simuliert wird, erwies sich dies als Problem. Die Lösung dieses Problems war, dass die Stapler nicht nur als Transportmittel von Agenten gesehen wurden, sondern wie Maschinen. So ergriff ein Stapler die Palette und brachte sie zu der jeweiligen Abteilung und nach dem dieser Arbeitsschritt des Staplers vollbracht war, wurde dieser freigegeben (released) und konnte die nächste Arbeit verrichten. Die Abfolge der Methoden, um einen Produktionsablauf zu generieren, ist in *Abbildung 6.8* zu sehen.

6.5.4 Statistiken

In diesem Abschnitt handelt es sich um die Statistiken, die in die Simulation eingefügt werden, um Echtzeit Daten der Produktion wiederzugeben. Diese berechnen statistische Informationen, wie zum Beispiel den Mittelwert, das Minimum und Maximum über eine Reihe von Datenproben. Zudem besteht die Möglichkeit verschiedene Arten von Diagrammen einzufügen. Die sind Balkendiagramme, Stapeldiagramme, Kreisdiagramme, Zeitdiagramme, Zeitstapeldiagramme, Zeitfarbendiagramme, Histogramme und zweidimensionale Histogramme.

Die Datentypen können zum Beispiel spezifisch ausgewählt werden. Für die Auslastung der Maschinen wird eine Skalierung von null bis eins festgelegt und diese sind vom Datentyp *double*.

Die Funktion von Statistiken sind unterschiedlich. Dies hängt davon ab, ob es sich bei den übergebenen Daten als diskret oder kontinuierlich handelt. [11]

Kontinuierliche Daten

Der Wert der Daten bleibt in kontinuierlicher Zeit bestehen, ändert sich aber nur zu diskreten Zeitpunkten. Die Stichproben müssen mit aufsteigenden Zeitpunkten. Diese müssen mit aufsteigenden Zeitstempeln zu dieser Statistik hinzugefügt werden. Die Funktionen, die dafür verwendet werden, gehen davon aus, dass der letzte Wert hinzugefügt wird. Der Mittelwert des Datensatzes ist ein zeitlich gewichteter Wert. [17]

Diskrete Daten

Diese haben keine zeitliche Dauer. Sie treten als isolierte und diskrete Punkte in der Zeit auf. Der Mittelwert, der sich daraus berechnen lässt, ist die Summe der einzelnen Stichproben geteilt durch die Anzahl der Stichproben. [17]

6.6 Erweiterung der Simulation anhand Java

In diesem Abschnitt handelt es sich um die Erweiterung der Simulation mit der Programmiersprache Java. Ohne die Verknüpfung von Simulation und Programmiersprache ist die Erstellung einer komplexen Simulation nicht möglich. Bei der Erstellung einer realen Welt im Simulationsmodell ist festzustellen, dass eine Notwendigkeit besteht für die Verwendung von Wahrscheinlichkeitsverteilungen, die Auswertung von Ausdrücken und Testbedingungen mit Eigenschaften von verschiedenen Objekten, um darauf die jeweiligen Algorithmen zu entwerfen. Das ist jedoch nicht nur durch die Verwendung von Grafiken möglich. Demnach enthält jede Simulationsmodellierungswerkzeug eine textuelle Skriptsprache.

Da Java eine höhere Programmiersprache ist, welche zu dem vollwertig objektorientiert ist, wurde sie für die Erweiterung der Simulationsmodelle in Anylogic ausgesucht. Zudem wird ein Modell, welches mit Anylogic erstellt wurde, vollständig in Java abgebildet. Dies bietet vor allem Entwicklern, die Erfahrung mit der Programmiersprache haben, den Vorteil, dass Probleme und Errors in der Simulation in der Main.java Klasse gelöst werden können.

6.6.1 Variablen

Für die Erstellung der Simulation wurde eine Variable benötigt, da ansonsten keine weitere Möglichkeit bestand, durch die Datenbank zu iterieren. Dafür wurde zudem eine weitere Spalte, in Form von einer ID Spalte, in die Datenbank eingefügt.

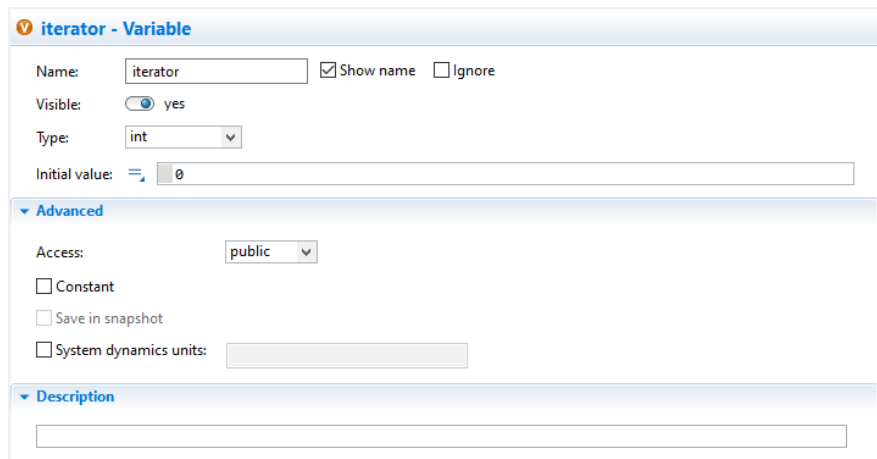


Abbildung 6.9: Variablen Eigenschaften in AnyLogic

Der Iterator ist ein Integer mit einem Default Wert von Null. Nach dem Start der Simulation werden Palleten (Agenten) generiert, wobei diese die jeweiligen Produktionsteile mit der dazugehörigen Artikelnummer sind.

```
SelectQuery sq = selectFrom(auftrag1)
    .where(auftrag1.id.eq(iterator));
if(sq.hasResults()) {
    agent.set_articleNumber(sq
        .firstResult(auftrag1.article_number));
    agent.initializeDepartments();
    iterator++;
}
```

Abbildung 6.10: Datenbank

In *Abbildung 6.10* ist der Java Code zu sehen, der die Palleten, mithilfe der Datenbank generiert. In jeder Iteration wird die jeweilige ID in der Datenbank mit dem Iterator verglichen und die dafür verwendete Artikelnummer wird dann aus der Datenbank entnommen. Die Artikelnummer wird für die Verzögerung (Delay) benötigt. Anhand der Artikelnummer werden die Laufzeiten abgefragt, die für die jeweiligen Produktionsteile benötigt werden. Dieser Vorgang ist in der nächsten Darstellung zu sehen. Das Beispiel in dieser Abbildung ist die Zeit, die die Abteilung für das Laserschneiden benötigt. Zudem wurde eine if-Schleife eingebaut, die die `NullPointerException` abfängt, wenn die Artikelnummer gleich null ist. Dies geschieht durch `hasResults()` Methode.

Abbildung 6.11: Laufzeitabfrage für das Laserschneiden

6.6.2 Funktionen

Für die Erstellung von komplexen Simulation, werden Funktionen benötigt. Die erste Funktion ist die *resetStats()* Funktion. Diese wird für das Zurücksetzen der Statistiken nach jedem Start angewendet, damit vorherige Ergebnisse die Auswertung nicht verfälschen.

Die *check()* Funktion ist für die korrekte Auswahl von Enter-Blöcken zuständig. Die erste Version der Simulation war nicht realistisch. Der Grund dafür war, dass die Logik eine Schleife war, die alle Abteilungen durchgegangen ist. Die Schleife ist in *Abbildung 6.12* zu sehen.

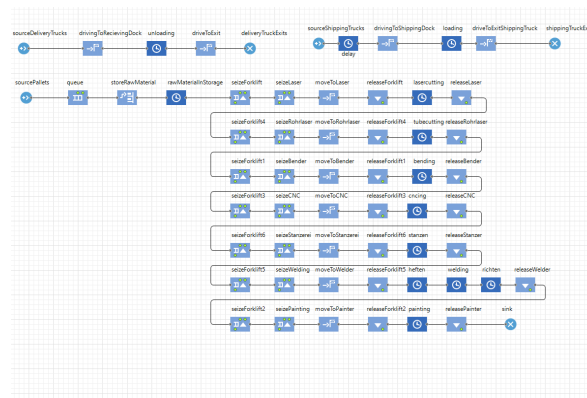


Abbildung 6.12: Logik der Simulation nach zwei Monaten

Der Agent wird durch alle Abteilungen gebracht, obwohl die dazu abgefragte Laufzeit gleich null war. Dies hat dazu geführt, dass die Abteilungen, trotz der nicht Benutzung, eine Auslastung hatten. Diese war vergleichsweise gering. Jedoch war diese Lösung nicht zufriedenstellend. Daraufhin wurde die Logik verändert und das Problem wurde durch Enter- und Exit-Blöcke behoben.



Abbildung 6.13: `check()` Funktion

Anhand der `check()` Funktion besteht die Möglichkeit die gewünschten Enter- oder Exit Blöcke auszuwählen und somit das obige Problem zu beheben. Die Funktion wird beim Verlassen eines jeden Exit-Blocks aufgerufen, um einen Agenten in die richtige Abteilung zu schicken. Die Funktion führt in jeder Iteration eine Kontrolle durch, ob der Wert der jeweiligen Abteilung in der Datenbank größer null ist und überspringt alle Abteilungen deren Werte kleiner gleich null sind.

Die Abteilungen für jeden Agenten werden in der `departments` Collection initialisiert, anhand der `initializeDepartments()` Funktion.

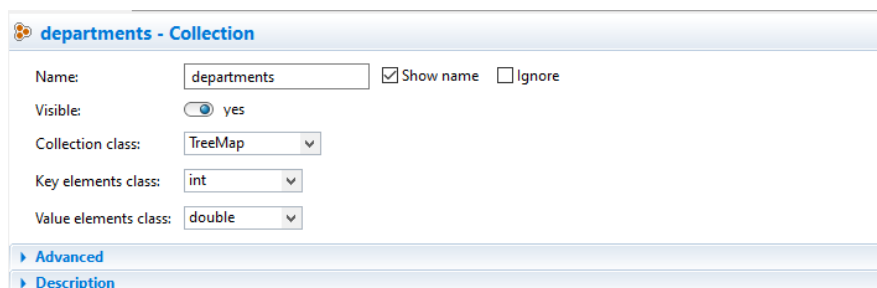
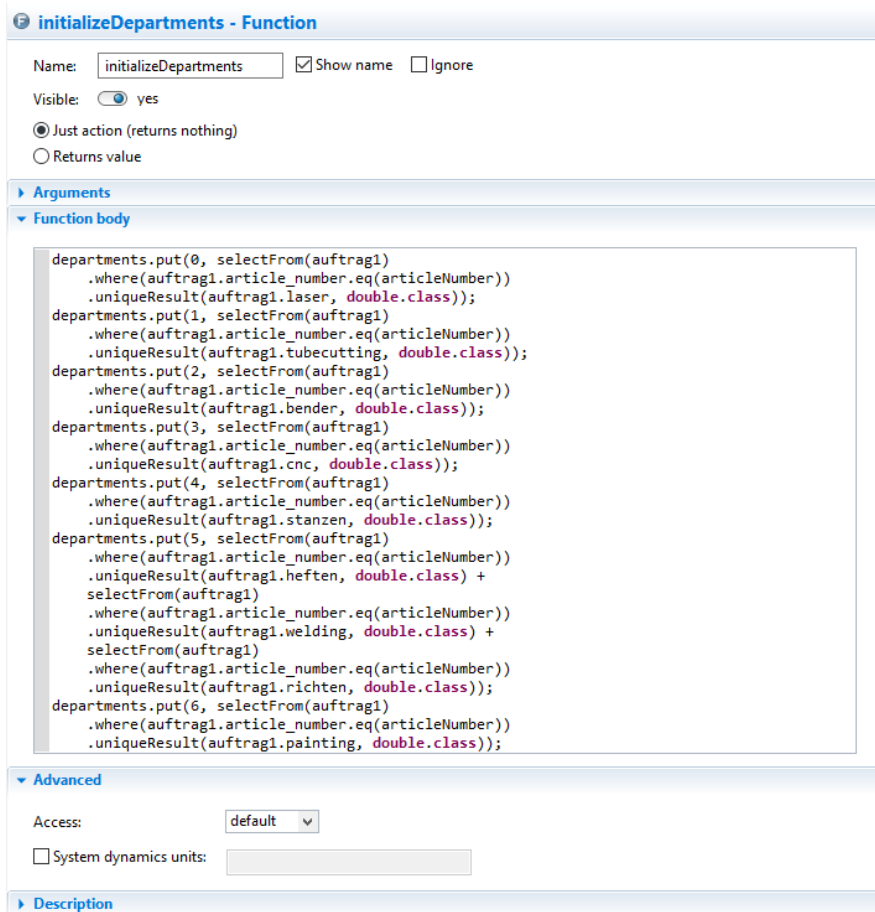


Abbildung 6.14: Angelegte Collection für Abteilungen

Die *Abbildung 6.14* zeigt die Collection. Die beinhaltet die Schlüssel (Key) und Werte (Value) Elemente. In *Abbildung 6.15* wird die `TreeMap` initialisiert. Die Schlüssel Elemente werden als Integer Werte initialisiert und die Werte Elemente werden anhand einer Query von der Datenbank abgefragt.

Abbildung 6.15: *initializeDepartments()* Funktion

6.6.3 Parameter

Eine weitere Komponente der Simulation sind Parameter. Parameter werden häufig für die Darstellung einiger Merkmale des modellierten Objektes verwendet. Parameter und Variablen unterscheiden sich an einem Punkt. Eine Variable repräsentiert einen Modellzustand und kann sich während der Simulation ändern. Ein Parameter wird für die statische Beschreibung von Objekten verwendet. Zudem sind Parameter meist konstante Werte. Eine Veränderung erfolgt nur, wenn das Verhalten des Modells angepasst werden soll. [11]

Ein Anwendungsbeispiel für Parameter ist die Rüstzeit. Diese beträgt laut dem Geschäftsführer 15 Minuten. Dies gilt im Durchschnitt für jede Abteilung. Demnach wurde der Parameter `setupDelay` erstellt mit einem Standardwert von 15.

6.6.4 Collections

Für die Initialisierung der Abteilungen wird eine Collection benötigt. Eine Collection ist eine Sammlung von Objekten. Collections werden zur Definition von Datenobjekten verwendet, die mehrere Elemente zu einer Einheit zusammenfassen. Ein weiteres Anwendungsbeispiel ist

die Speicherung, das Abrufen und Bearbeiten von aggregierten Daten. [11]

In der *Abbildung 6.14* ist die Darstellung der Collection, die für die Erstellung einer Sammlung von Abteilungen in dem Produktionsablauf benötigt wird.

6.7 Erfahrung

In diesem Abschnitt werden die Erfahrungen geschildert, die während dem dreimonatigen Zeitraum gesammelt wurden. Diese wurden nur mit großem Aufwand gelöst, da diese Probleme meist spezifisch sind und die Anzahl der Antwortmöglichkeiten im Internet sehr gering sind. Ein großes Hilfsmittel für die Erstellung der Simulation war die Bibliothek (Any Logic Help) der Simulationssoftware, der Support und die wöchentlichen Meetings mit dem Prof. Dr. Jörg Hähner.

Die Simulationssoftware bietet verschiedene Anwendungsbereiche an für die Erstellung von Simulationen. Diese sind:

- Process Modelling (Prozessmodellierung)
- Material Handling (Fördertechnik)
- Rail (Schienen)
- Road Traffic (Straßenverkehr)
- Fluid (Flüssigkeiten)

Für diese Arbeit wurde die Process Modelling Palette verwendet. Zu Beginn der Simulation wurde ein einfaches Modell erstellt, um ein gewisses Grundwissen aufzubauen. Dazu dienen die Tutorials und Beispiele, die von der Simulationssoftware zur Verfügung gestellt werden. Nach dem Erarbeiten eines Grundwissens wurde mit der Erstellung der Simulation begonnen. Das Einfügen von Maschinen, Hallen, Gebäude, Wänden und anderen Objekten stellte keinerlei Probleme dar.

Im nächsten Schritt wurden Knoten, Pfade, Pallet Racks und Rectangle Nodes eingebaut. Diese mussten stets miteinander verbunden sein, damit diese als ein gesamtes Netzwerk miteinander arbeiten können. Schon nach diesen einfachen Schritten fingen Probleme an, da ein Pfad einmal durch den Gang eines Pallet Racks verlaufen musste, damit die Simulation starten konnte.

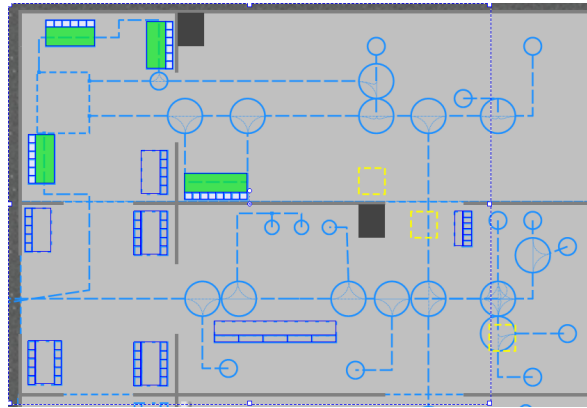


Abbildung 6.16: Pallet Racks im Netzwerk

In der *Abbildung 6.16* ist ein Netzwerk zu erkennen, in dem die Pallet Racks markiert sind. Da ist deutlich zu sehen, dass Pallet Racks vorhanden sind, dessen Gänge grün markiert werden. Diese können während der Simulation angesteuert und als Palettenregal verwendet werden. Wenn die anderen Paletten verwendet werden wollen und als Objekt übergeben werden, so ist das Starten der Simulation nicht möglich.

Im Laufe der Zeit wurde die Simulation immer größer und komplexer. Nach dem Implementieren der verschiedenen Abteilungen, in Form von Ressource Pools, Point Nodes, Objekten und Pfaden, entstand ein weiteres Problem, dass wiederum einige Zeit beansprucht hat, da zu diesem Problem keine Lösungen vorhanden waren. Dieses war das Transportieren der Agenten (Paletten) anhand eines anderen Agenten (Stapler) erwies sich als Problem. Die Tutorials bestehen nur aus einer Abteilung oder Maschine und weisen keinerlei Komplexität auf. Da in dieser Produktion sieben verschiedene Abteilungen vorhanden waren, war das Lösen wie in den Tutorials und Beispielen nicht möglich.

Zu Beginn wurde die Methode `moveTo` verwendet und dadurch entstand die Problematik, dass sich die Paletten selbstständig in den Abteilungen hin und her bewegten. Für die Erstellung einer sauberen und realistischen Simulation wurde demnächst die Methode `RackPick` verwendet. Diese jedoch verursachte das Problem, dass die Paletten mit den Rohmaterialien nur von der PalletRack in die nächste Maschine gebracht werden kann und für den Transport davon immer PalletRacks benötigt werden, um die Paletten mit dem Stapler aufzunehmen und zu bewegen. Demnach wurde ein anderer Ansatz realisiert, indem die Stapler als Maschinen wahrgenommen werden und erst dann an ihren Parkplatz wieder zurück können, wenn die Arbeit verrichtet ist. Der Ressource Pool *stapler* wird aufgenommen und als Transportmittel verwendet, um die Materialien von einer Abteilung zu der Nächsten zu bringen. Nach dem Erfüllen der Schritte werden die Stapler *released* und bewegen sich an den Staplerparkplatz.

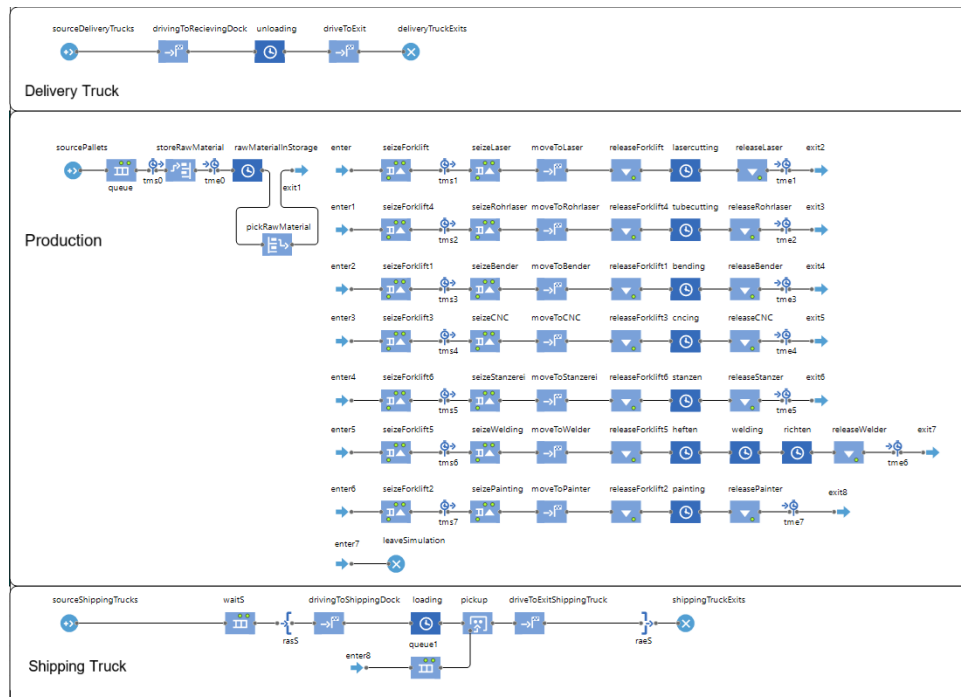


Abbildung 6.17: Logik der Simulation

Die Logik wurde im Laufe der Arbeit stets bearbeitet. Die *Abbildung 6.12* zeigt eine Lösung, die bis zur Analyse der Simulation verwendet wurde. Als es zur Analyse des Produktionsflusses kam, entstand das Problem, dass Abteilungen eine Auslastung erzeugten, obwohl die SQL Query den Wert null lieferte. Einen Ansatz für die Lösung zu diesem Problem war kaum möglich. Nach ausgiebiger Recherche und Testversuchen wurden Exit- und Enter Blöcken angewendet, die anhand von Funktionen angesteuert werden können. In *Abbildung 6.17* ist die endgültige Logik der Simulation zu sehen.

Ein weiteres Problem war das Lösen eines Errors, welcher stets am Ende der Simulation auftauchte, wenn der *sourceDeliveryTrucks* auftauchte, um neue Paletten zu liefern. Trotz des Erstellen eines Rack Systems wurde stets ein Fehler geworfen, dass keine leeren Zellen in den Palettenregalen vorhanden sind, obwohl diese leer sind. Die Paletten (Agenten) werden durch die *moveTo* Methode zu der jeweiligen Abteilung gebracht. Die Agenten wird von dem *Exit1* Block aufgenommen und die *check()* Funktion bestimmt, was mit den Paletten geschieht. Zu einem späteren Zeitpunkt wurde festgestellt, dass die Zelle von den Agenten reserviert wird, sobald dieser den *storeRawMaterial* Block betritt. Auch wenn der Agent zu einem späteren Zeitpunkt anhand eines Exit Blocks herausgenommen wird, ist die Zelle immer noch belegt, obwohl der Agent die Zelle physisch verlässt. Um die Reservierung freizugeben, musste ein *RackPick* Block eingefügt werden, um die Zellen von den Agenten freizugeben.

Abschließend ist zu erwähnen, dass es sich bei der Lizenz um eine Personal Learning Edition handelt. Ein Beispiel woran dies zu erkennen ist, dass eine Einschränkung bei der Anzahl von Agenten vorhanden ist. Folglich bestand nicht die Möglichkeit einen Agenten für die Stanzerei zu erstellen. Demnach wurde für die Stanzerei lediglich ein Objekt angelegt.

7 Auswertung

Die Auswertung der Simulation ist der nächste Schritt dieser Arbeit. Dieses Kapitel ist eines der wichtigsten Abschnitte der Arbeit, da das Unternehmen den Prozess, die Effizienz und wichtige Parameter für die Abfertigung ausgeben und analysieren kann. Die Auswertung geschieht anhand von Diagrammen und Histogrammen. Diese bieten die Möglichkeit Auswertungen grafisch darzustellen und zu analysieren.

Demnach wird ein realer Auftrag des Unternehmens simuliert und die dafür benötigte Zeit erfasst und festgehalten, um verschiedene Konfigurationen miteinander zu vergleichen. Die dafür verwendete Zeit ist die Gesamtlaufzeit für die Beendigung des Auftrages. Anylogic zeigt analog zur Simulation die Zeit ab dem Start der Simulation an. Wenn alle Aufträge beendet worden sind, wird die *finishSimulation()* Methode aufgerufen. Dies geschieht anhand einer Funktion, die die benötigte Anzahl an Aufträgen vergleicht. Nach dem Abarbeiten aller Aufträge und Arbeitsschritte wird die Simulation beendet.

Das Unternehmen kann aufgrund von Erfahrung und der automatischen Berechnung der Laufzeit der Maschinen die Dauer abschätzen, die der Auftrag benötigt. Danach können die zwei Zeiten in einem Vergleich gegenüber gestellt und Gründe für den Zeitunterschied ermittelt werden. Dass die Simulation in den meisten Fällen eine bessere Zeit erzielt ist sehr wahrscheinlich. Das liegt daran, dass die Simulation den best-case simuliert. Der Grund dafür ist, dass in der Simulation keine Fehler und Problematiken entstehen. Das geschieht in der Realität etwas öfter. Eines der Gründe ist, dass ein großer Teil der Arbeit von Menschen verrichtet wird und diese Fehler machen. Dies ist jedoch ein stochastisches Problem. Um dieses Problem zu lösen, wäre es möglich stochastisch Fehler oder Wartezeiten anhand von Delays einzufügen. Zum Beispiel, dass in der Simulation in in regelmäßigen Abständen von drei Stunden ein Fehler mit der Wahrscheinlichkeit von fünf Prozent auftritt, der in der jeweiligen Abteilung eine Verzögerung von 30 Minuten verursacht.

Anschließend können nach der Veränderung der Eigenschaften und der Konfiguration der Simulation die jeweils erstellten Grafiken miteinander verglichen werden, um daraus dementprechende Schlussfolgerungen zu ziehen.

7.1 Aktueller Produktionsablauf

Der aktuelle Produktionsablauf des Unternehmens ist auf Basis von einem dezentralen System. Es ist kein Zentrallager vorhanden, in dem die fertigen Produktionsteile gelagert werden. Der Produktionsablauf beginnt mit dem Erstellen des Auftrages. Dieser beinhaltet die Auftragsnummer, Stückzahl und die jeweils benötigten Arbeitsschritte. Ein Großteil der Materialien werden als Bleche angeliefert. Diese werden in der Simulation anhand dem *sourceDeliveryTrucks* angeliefert. Die gelieferten Rohmaterialien werden mithilfe von Staplern ausgeladen

und in das jeweilige Paletttenregal transportiert. In diesem werden die Paletten für die Verarbeitung gelagert. Nach der Lagerung der angelieferten Rohmaterialien wird ein Delay gestartet, bevor die Produktion startet, da die angelieferten Rohmaterialien eine bestimmte Zeit im Paletttenregal bleiben und meist nicht direkt verarbeitet werden. Für die Implementierung des Delays (Verzögerung), bevor die Produktion anfängt, wird die Methode *triangular(int1, int2, int3)* verwendet. Demnach wird für den Delay ein Wert von den drei angegebenen Integer Werten verwendet, welcher zudem mit den Attributen min, sec oder h versehen werden. Die Rohbleche müssen immer zu aller erst anhand der Lasermaschine zugeschnitten werden. Zudem werden Rohre angeliefert diese werden ebenfalls zuerst an die Rohrlasermaschine gebracht und verarbeitet. Die Maschinen und verschiedenen Abteilungen sind in den *Abbildungen 7.1 - 7-3* zu sehen. Die Grafik stellt dreidimensionale Ausschnitte der Hallen und dem Firmengelände dar, die von der Simulation entnommen sind.

Danach werden Teile, die zugeschnitten und verarbeitet wurden, an die jeweiligen Abteilungen gebracht. Der Ablauf wird stets vorgegeben und abgearbeitet, weshalb die Reihenfolge eine wichtige Rolle spielt (Wahrscheinlichkeitstheorie). Nach dem Abarbeiten der Arbeitvorgänge werden dann zu aller letzt die gefertigten Teile in die Pulverei gebracht, welcher oftmals der letzte Schritt in der Produktion ist, bevor die Artikel ausgeliefert werden.

Auf die Nachfrage an den Inhaber, ob das Unternehmen eine Vorgehensweise in der Produktion oder in der Lagerung habe, wurde als Antwort erhalten, dass es keinen Plan gäbe und die Teile unter anderem zufällig gelagert werden. Demnach wurde auch die Frage, ob sich eine saisionale Lagerung lohnen würde, beantwortet.

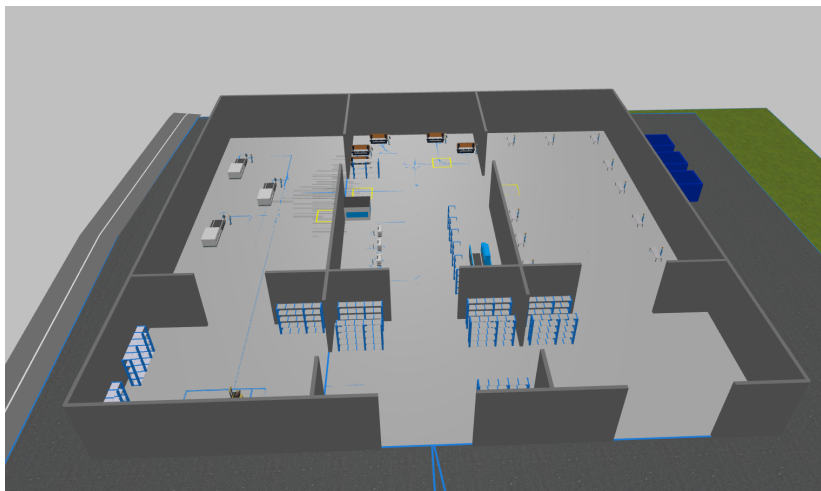


Abbildung 7.1: Draufsicht der ersten Produktionshalle

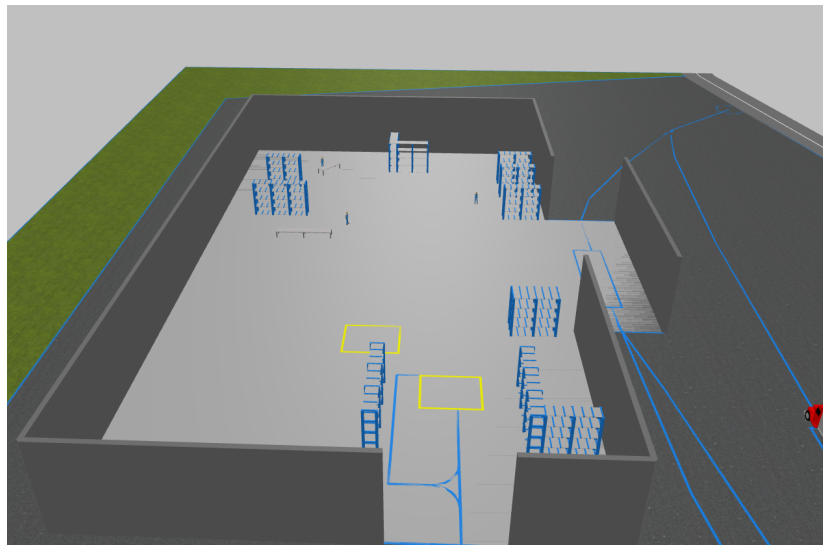


Abbildung 7.2: Draufsicht der zweiten Produktionshalle

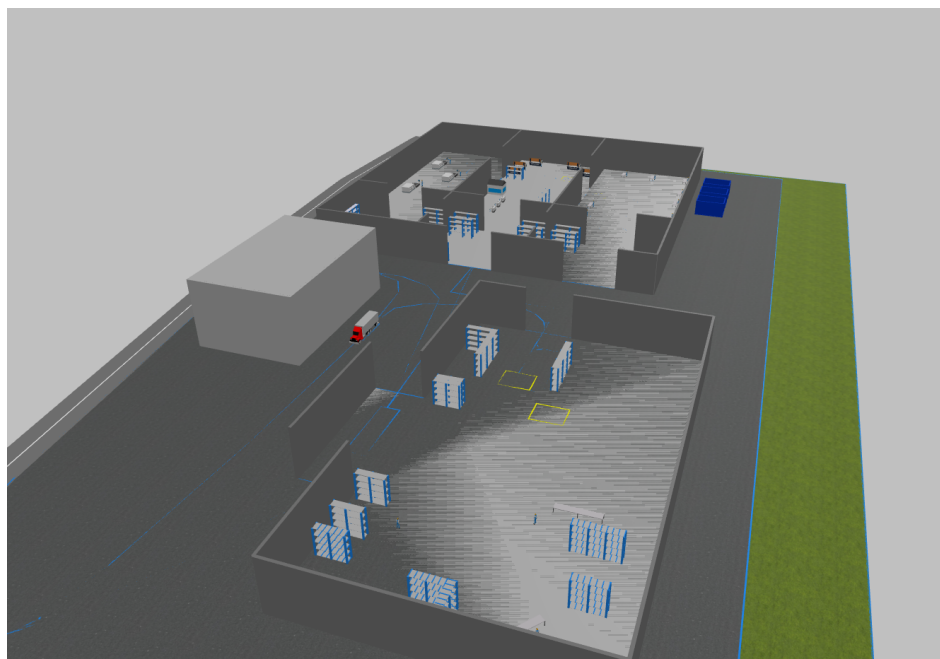


Abbildung 7.3: Draufsicht des kompletten Firmengeländes

7.1.1 Analyse des aktuellen Produktionsablaufes

In dem vorliegenden Abschnitt handelt sich um die Analyse des Produktionsablaufes. Die Analyse erfolgt anhand der Simulation und den erstellten echtzeit Diagrammen. Diese enthalten die Auswertung der für die Produktion bedeutenden Parameter. Dazu zählen zum Beispiel die Auslastung der Maschinen und die Zeit, die für einen jeweiligen Auftrag benötigt wird. Für

die Analyse des Ablaufes werden mehrere Aufträge verwendet, um eine realistische Simulation zu gewährleisten. Des weiteren musste für die Erstellung der Simulation eine Datenbank angelegt, die die benötigten Daten für den Ablauf enthält. Während der Simulation wurde, außer der zwei- oder dreidimensionalen Ansicht, die Logik beobachtet. AnyLogic ermöglicht dem Nutzer während der Simulation Echtzeitwerte zu beobachten.

Die *Abbildung 7.4* zeigt die Ausgabe von Werten der zu simulierenden Produktion an jedem verfügbaren Block. Das ermöglicht dem Nutzer die Simulation währenddessen zu verfolgen und Fehler schneller zu erkennen. Dieses Verhalten ist ähnlich dem eines Sensors. Denn die Ausgabe dieser Werte erlaubt dem User die Überwachung von physikalischen und ökologischen Zuständen. [22]

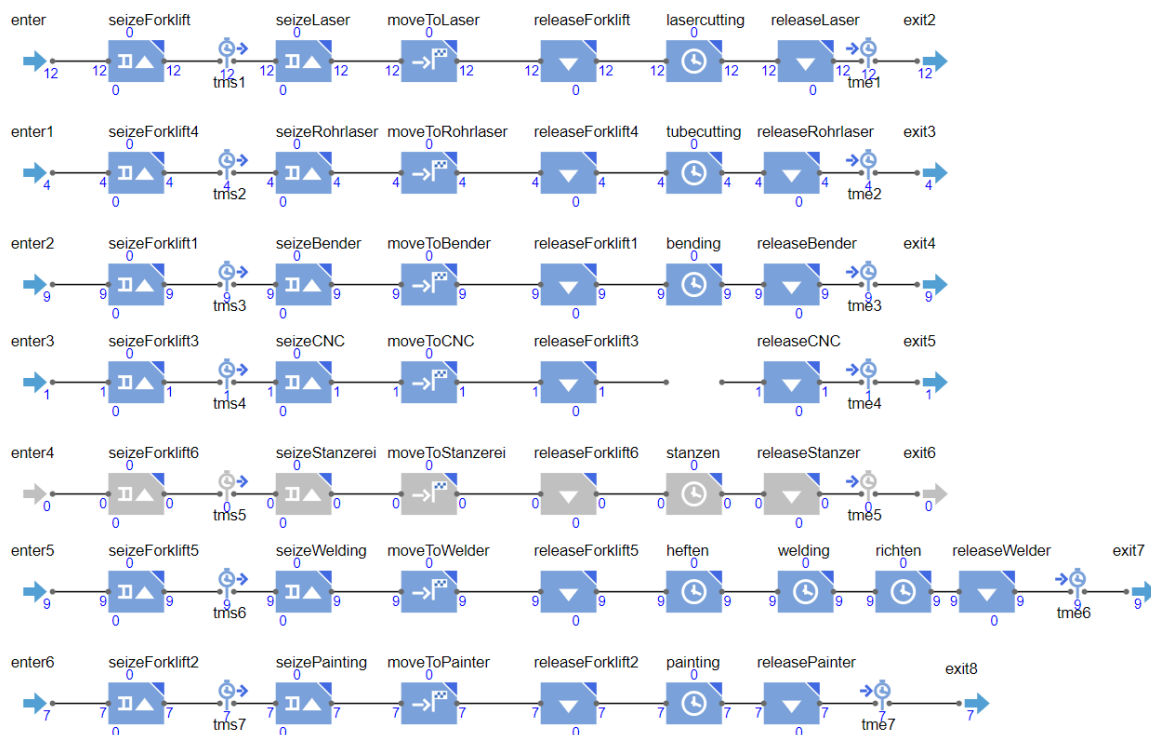


Abbildung 7.4: Echtzeitausgabe von Werten

Die Datenbank beinhaltet alle Arbeitsvorgänge des Unternehmens. In *Abbildung 7.5* wurden die Arbeitsvorgänge als Spaltennamen versehen und die Zahl in diesen jeweiligen Zellen ist die Zeit der Produktionsteile in Minuten, die es in der entsprechenden Abteilung benötigt. Ein weiteres Attribut ist die Kapazität (quantity). Dieses beinhaltet die Anzahl der Werkstücke, die hergestellt werden müssen. Die Daten werden anhand von Java Befehlen mit der Simulation verknüpft und abgearbeitet. Die Datenbank beinhaltet multiple Aufträge, deren echte Laufzeit wird in der Datenbank der Firma festgehalten, um damit die Kosten dafür zu berechnen.

	id	article_number	quantity	laser	bender	cnc	heften	welding	richten	stanzen	tubecutting	painting
1	0	9,002,081	20	48	0	0	0	0	0	0	0	0
2	1	90,023,056	13	26	13	0	26	39	0	0	0	0
3	2	90,023,055	13	26	13	0	26	39	0	0	0	0
4	3	90,023,151	13	0	0	0	78	156	130	0	0	0
5	4	6,156,043	30	90	78	0	0	600	0	0	0	102
6	5	6,156,140	108	255.96	108	0	0	1,944	0	0	0	302.4
7	6	6,406,499	15	0	0	30	0	0	0	0	0	12
8	7	90,014,714	20	76	24	0	0	0	0	0	0	60
9	8	6,611,393	398	704.46	159.2	0	0	0	0	0	0	796
10	9	9,002,071	120	0	0	0	0	1,560	0	0	540	960
11	10	90,009,920	60	252	210	0	0	2,400	0	0	0	300
12	11	90,022,415	100	0	0	0	0	0	0	0	140	0
13	12	90,001,279	20	8	6	0	0	0	0	0	0	0
14	13	6,158,398	25	47.5	0	0	0	300	0	0	30	0
15	14	6,158,399	25	47.5	0	0	0	375	0	0	30	0
16	15	6,153,365	20	6.8	6.67	0	0	0	0	0	0	0
17	16	6,114,516	30	40.2	5	0	0	0	0	0	0	0
*												

Abbildung 7.5: Eingfügter Auftrag in die Datenbank

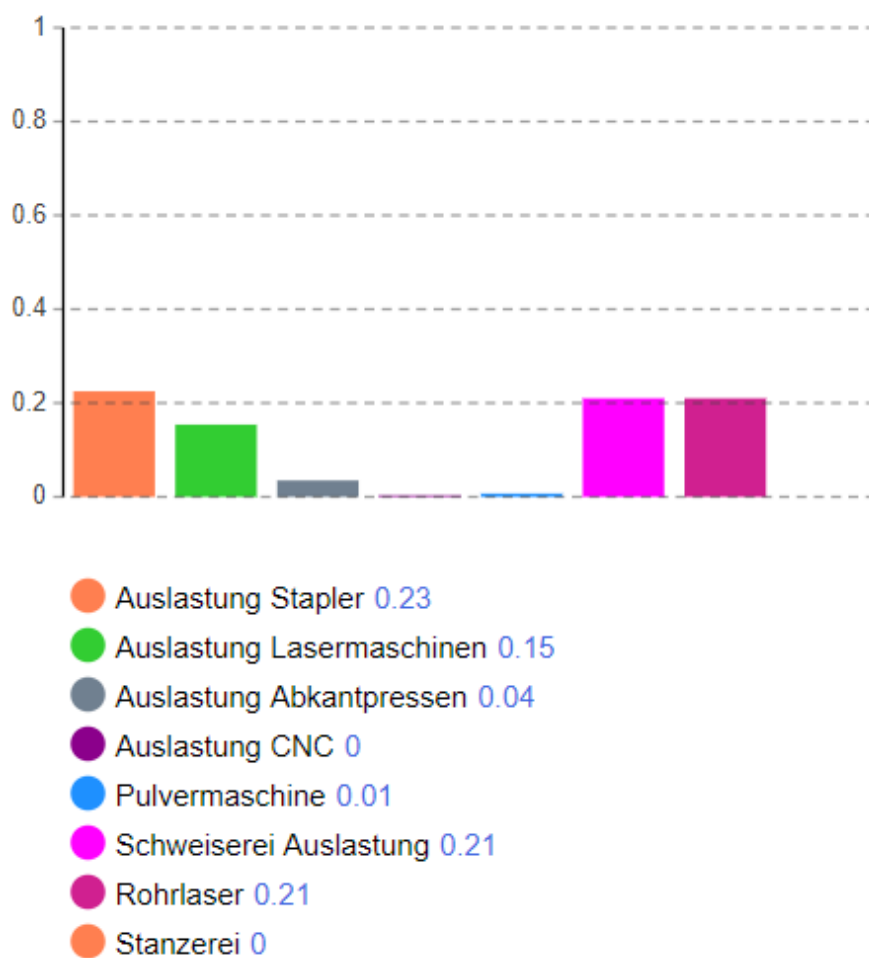


Abbildung 7.6: Auslastungen nach ungefähr 59 Stunden

Nach dem Durchlaufen des Auftrages wird zunächst die Auslastung der Maschinen ausgewertet. Die *double utilization()* Methode gibt die Auslastung der Resource Pools (hier: Abteilungen) aus. Der zurückgegebene Wert ist die Durchschnittsauslastung über allen Maschinen in den Abteilungen, vom Start bis zum Ende der Simulation.

Nach einem ungefähr 58,8 stündigen (211.869,36 sec) Durchlauf sind alle Produkte fertig gestellt und es fällt auf, dass keine Abteilung eine hundertprozentige Auslastung aufweist. Die höchsten Auslastungen erreichen die Stapler, Schweißerei und der Rohrlaser. Wie in der Datenbank zu sehen ist, wird für die hierfür benötigten Aufträge keine Stanzerei benötigt, weshalb die Auslastung gleich null ist. Die Auslastungen der CNC Abteilung ist ebenfalls sehr gering. Die Werte, die in den Diagrammen ausgegeben werden, wurden auf zwei Stellen nach dem Komma gerundet. Betrachtet man die ungerundeten Werte so erhält man:

- Auslastung Stapler 0.225
- Auslastung Lasermaschinen 0.154
- Auslastung Abkantpressen 0.035
- Auslastung CNC 0.003
- Auslastung Pulvermaschine 0.007
- Auslastung Schweißerei Auslastung 0.21
- Auslastung Rohrlaser 0.21
- Auslastung Stanzerei 0

An jeder Maschine und Abteilung sind mehrere Arbeitskräfte beteiligt, was wiederum darauf schließen lässt, dass diese deshalb nicht arbeiten können. Problematisch ist das für das Unternehmen, denn diese bezahlen ihre Arbeitskräfte auf Stunden, die diese erbringen und nicht auf die Anzahl der Produkte, die gefertigt werden. Deswegen sollte das Unternehmen eine höhere Auslastung generieren, um ihren Gewinn zu erhöhen und die Ausgaben zu verringern.

Ein weiterer Parameter, der benötigt wird, ist der prozentuale Anteil der jeweiligen Abteilungen in der Simulation. Für die Erstellung und Berechnung der Auswertung für diesen Parameter werden Zeitstempel in die Logik implementiert.

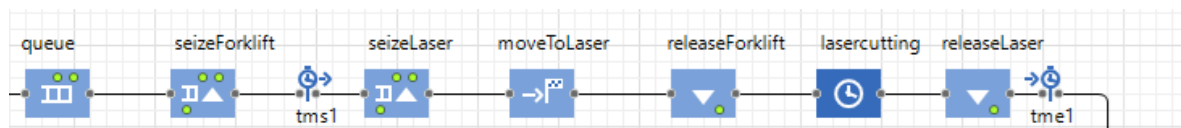


Abbildung 7.7: Time Measure Start / - End für Zeitmessungen

Dies geschieht anhand der Time Measure Start und Time Measure End Methode. Die Time Measure End Methode bekommt einen oder mehrere Time Measure Start Block als Parameter übergeben. Die *Abbildung 7.7* zeigt die Zeitenmessung, die in die Logik der Laserabteilung eingesetzt wurde. Dabei wird die Zeit gemessen, in dem der Stapler das Werkstück in die Laserabteilung bringt, bis die Abteilung den Agenten (Paletten) loslässt (releaseLaser) und somit mit dem Produktionsstück fertig ist. Dabei beginnt die Messung mit *tms1* (Time Measure

Start) und endet mit *tme1* (Time Measure End). Für die Berechnung des Durchschnittes wird folgende Berechnung durchgeführt:

tme0.distribution.mean() / minute()

Dabei wird der Mittelwert der Verteilung von der Zeit in der jeweiligen Abteilung berechnet. Dieser Wert wird dann durch die Zeit in Minuten geteilt.

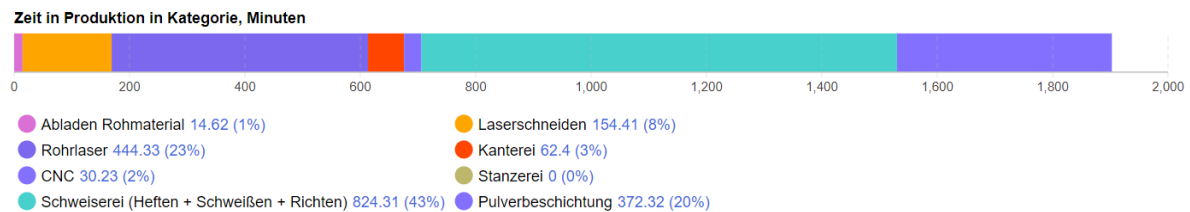


Abbildung 7.8: Verteilung in den Abteilungen

Die *Abbildung 7.8* zeigt die Verteilung der Produktion in den verschiedenen Abteilungen mit der dazugehörigen Zeit in Minuten. Dabei ist deutlich zu erkennen, dass die Abteilung der Rohrlaser am meisten Zeit in Anspruch nimmt. Dies liegt daran, dass in dieser Abteilung eine Maschine betrieben wird, um somit die Zeit und der prozentuale Anteil höher ist. Nach dem Begutachten der Datenbank fällt auf, dass die Laser- und Schweiserei-Abteilung am meisten Zeit benötigen werden. Das wird jedoch zum Beispiel anhand der hohen Anzahl von Schweisern oder Lasermaschinen gelöst. Hätte man beispielsweise in jeder Abteilung eine Maschine, wären die Zeiten für die Schweiserei und die Laserabteilung deutlich höher. Da die Aufträge jedoch auf mehrere Maschinen oder Personen aufgeteilt werden, wird die dafür benötigte Zeit verringert.

7.2 Optimierungsansätze von der Seite des Unternehmens

Da das Unternehmen die Produktionshallen neu gebaut hat, hat das Unternehmen bei Strukturierung der Produktion Gedanken gemacht, um einen effizienten Produktionsablauf zu gewährleisten. Ein Ansatz für die Optimierung des Produktionsflusses, ist die Errichtung eines Zentrallagers. Das wird in der Simulation implementiert und getestet. In *Abbildung 7.9* ist das Zentrallager zu sehen.

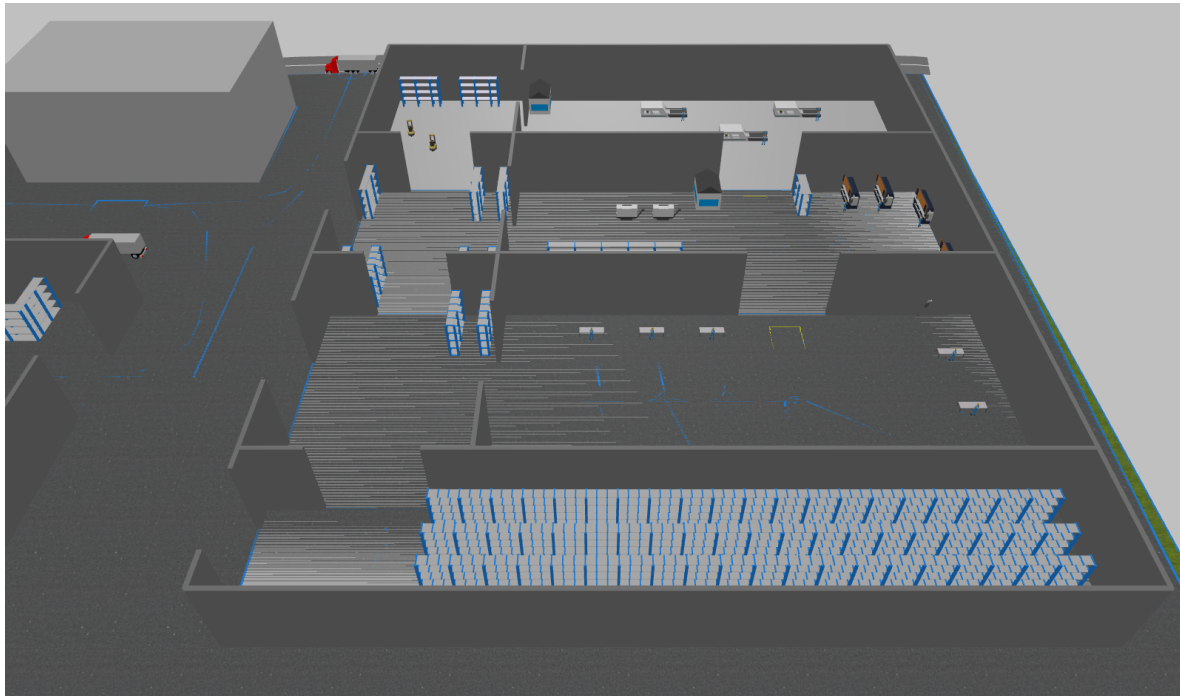


Abbildung 7.9: Implementation des Zentrallagers

Der Bau des Lagers wird an die Halle 1 angeschlossen. Für die Simulation werden PalletRacks und Pfade eingefügt. Zudem muss die Logik angepasst werden.

7.2.1 Analyse Simulation mit einem Zentrallager

Lohnt sich die Investition im Hinblick darauf, dass sich der Ablauf in Zukunft beschleunigt, da dies eine kostenintensive Investition ist. Diese Investition muss auf einen längeren Zeitraum getestet werden.



Abbildung 7.10: Verteilung der Auslastungen nach Einfügen eines Zentrallagers

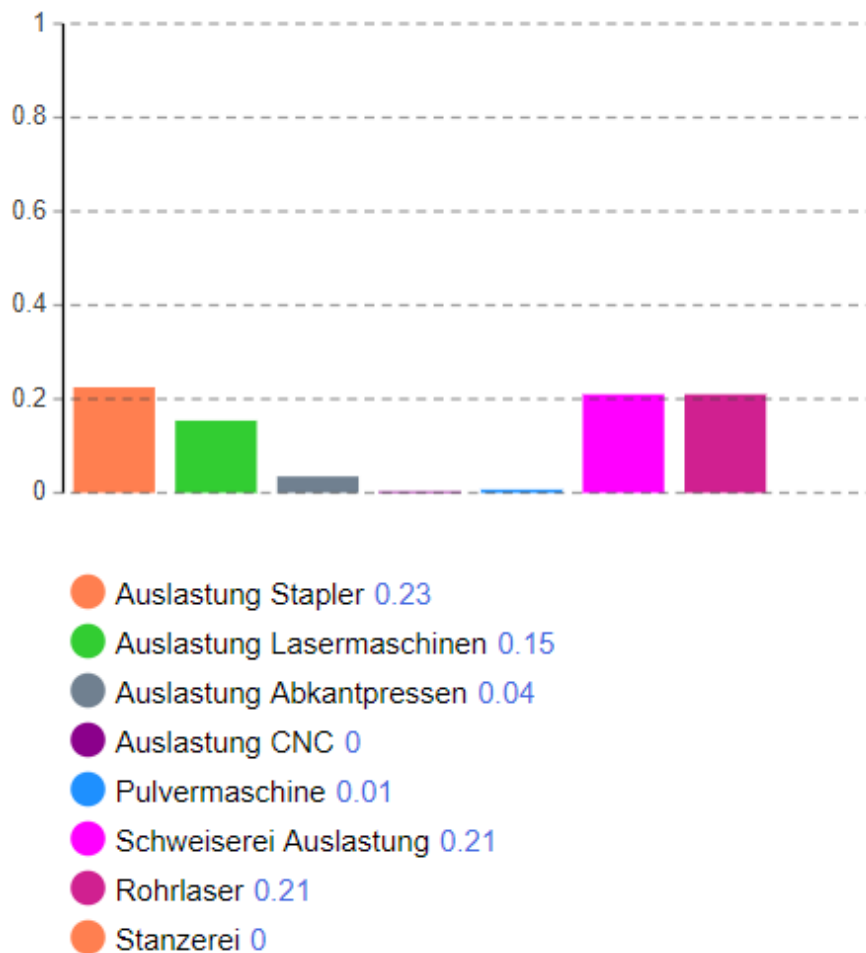


Abbildung 7.11: Auslastung Simulation nach Einfügen eines Zentrallagers

Um die Simulationen miteinander zu vergleichen, bearbeitet diese Simulation die identische Datenbank, wie die Simulation in *Abschnitt 7.1.1*. Aus den *Abbildungen 7.10/.11* ist deutlich erkennbar, dass die Implementation eines Zentrallagers in das System keine messbare Verbesserung erzeugt. Die Werte der zwei verschiedenen Simulationen, mit und ohne ein Zentrallager, sind beinahe identisch. Trotz aller dem hat der Bau eines Zentrallagers einige Vorteile. Einer dieser wesentlichen Vorteile ist die hohe Lieferbereitschaft. Betrachtet man die aktuellen Geschehnisse, ist der Mangel an Warenbeständen heutzutage ein großes Problem. Aufgrund dessen kommt es immer wieder zu Lieferengpässen. Diese und weitere Probleme wären anhand eines Zentrallagers behebbar.

7.3 Optimierung des Produktionsablaufes

In diesem Absatz geht es um die Auswahl eines Optimierungsansatzes. Das Ziel ist es die Produktion auf langer Hinsicht zu optimieren und eine hohe Effizienz zu garantieren.

Zuerst muss überlegt werden, um welche Art von Problem es sich handelt. In dem Unternehmen sind mehrere Abteilungen und Arbeitsvorgänge vorhanden, die für die jeweiligen Produktionsteile abgearbeitet werden müssen. Dabei handelt es sich um eine Kombination aus mehreren Abteilungen, wobei jeder Durchlauf eine Reihe von Arbeitsvorgängen ist. Das ist gleichzusetzen mit einer Permutation, da es sich um eine Kombinationsproblem von einer Anordnung von von Objekten in einer bestimmten Reihenfolge handelt. Hierbei handelt es sich um eine Permutation ohne Wiederholung.

Ein Optimierungsansatz für das Lösen von diesem Problem wäre die Verwendung von Metaheuristiken. Jedoch erwies sich dies für die Anwendung in der Simulation als problematisch. Das Grund ist die Komplexität der Software. AnyLogic zählt zu den stärksten und komplexesten Simulationsframeworks, die es aktuell auf dem Markt gibt. Der kurze Zeitraum von drei Monaten ermöglichte eine solche Implementation nicht.

Nach dem Kontaktieren des Supports, wurde mitgeteilt, dass AnyLogic Optimierungsversuche, unter bestimmten Bedingungen, anhand der Beobachtung des Systemverhaltens, anbietet. Durch die Veränderung dieser Parameter kann das Modell getestet und optimiert werden.

7.3.1 Veränderungen an der Konfiguration

Ein Optimierungsansatz für die Verbesserung des Produktionsflusses ist die Veränderung der Parameter. Anhand der Simulation besteht die Möglichkeit Eigenschaften der Produktion zu konfigurieren und zu testen, ob diese Veränderung zu einer Verbesserung der Laufzeit führt. Nach einer minimalen Bearbeitung der Konfiguration fällt auf, dass die Verteilung der Abteilungen deutliche Veränderungen aufweist. Die Anzahl der Stapler, Rohrlaser und Schweißer wurde um einen Agenten erhöht. So kann die Zeit, die in der Abteilung der Rohrlaser benötigt wird, deutlich verringert werden. Das Erhöhen der Schweißereiabteilung um einen Agenten hat kaum Auswirkungen in den Statistiken bewirkt.

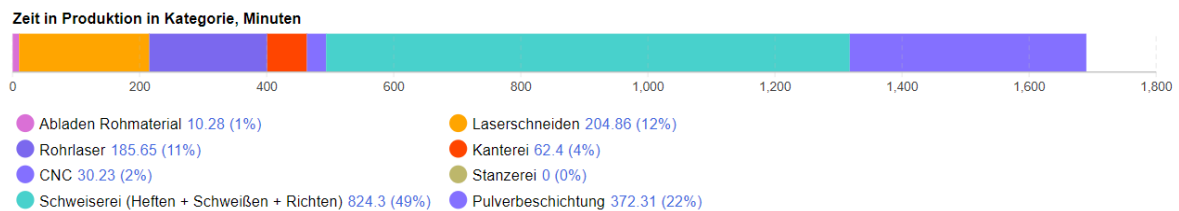


Abbildung 7.12: Verteilung der Abteilungen nach minimaler Veränderung

Es ist deutlich zu erkennen, dass sie die Verteilung verändert und die Auslastung der Maschinen verringert wird. Das Ziel eines Unternehmens ist stets die maximale Auslastung der Maschinen und die Steigerung der Produktdichte. Eine Verringerung der Auslastung trägt dazu bei, dass der Produktionsfluss in der Lage ist eine höhere Anzahl an Artikel zu produzieren.

- Auslastung Stapler 0.121

- Auslastung Lasermaschinen 0.164
- Auslastung Abkantpressen 0.038
- Auslastung CNC 0.003
- Auslastung Pulvermaschine 0.008
- Auslastung Schweißerei Auslastung 0.204
- Auslastung Rohrlaser 0.112
- Auslastung Stanzerei 0

Die veränderte Konfiguration der Parameter führte dazu, dass die Zeit für das Abarbeiten der Aufträge verringert wird. In *Abschnitt 7.1.1* wurde ermittelt, dass das System für die Aufträge eine Zeit von ungefähr 59 Stunden benötigt. Das neue System erreicht eine Zeit von ungefähr 55,2 Stunden. Betrachtet man die Gesamtlaufzeiten genauer so erhalten wir nach einer minimalen Veränderung der Parameter eine Zeitersparnis von 13.050 Sekunden. Die dadurch erreichte Verbesserung beträgt 3,63 Stunden. Somit wurde eine Verbesserung von ungefähr sechs Prozent erzielt.

7.3.2 Veränderung der Reihenfolge der Aufträge

Eine Annahme für die Optimierung des Produktionsflusses ist die Veränderung der Reihenfolge, in der die Aufträge abgearbeitet werden. Dafür wurden die Aufträge in der Datenbank nach deren Laufzeit aufsteigend sortiert. Damit können Aufträge, die schneller gefertigt werden können, aufgrund der geringeren Laufzeit, früher ausgeliefert werden.

Nach dem Beenden der Simulation fällt auf, dass die Gesamtlaufzeit der Simulation in einigen Fällen zu einer Verschlechterung und in einigen zu einer Verbesserung führt.

Für die Ermittlung der Zeit für jeden spezifischen Auftrag wird, wurde die Simulation erneut verändert. Dafür werden dem Agenten *Pallet* neue Variablen ein neuer Parameter übergeben. Nach dem Verlassen von einem *Pallet* Agenten aus dem Palettenregal wird der Variable *startTime* die aktuelle Zeit übergeben. Wobei *agent* der dabei verwendete Agenten Typ ist, der sich in dem aktuellen Block befindet.

```
agent.startTime = time();
```

Nachdem der *Pallet* Agent alle Arbeitsschritte durchlaufen hat, betritt dieser den *enter8* Block. Nach dem Eintreten des Agenten in den Block, wird die Zeit für berechnet, die ein spezifischer Auftrag bis zur vollständigen Beendigung benötigt.

```
agent.time = time() - agent.startTime;  
timePerTask.get(agent.palletID).setValue(agent.time);
```

Für die grafische Auswertung wird das Diagramm *timePerTask* erstellt. Nach der Vollendung von jedem Auftrag wird diesem Diagramm, anhand der ID des Auftrages, die Zeit übergeben, die für den Abschluss eines Auftrages benötigt wird.

Die *Abbildung 7.13* stellt die Verteilung der Zeit für jeden Auftrag dar. Anzumerken ist, dass die Namen der Balken statisch vergeben werden, da den Balken nur ein Titel und keine Variable zugeordnet werden konnte.

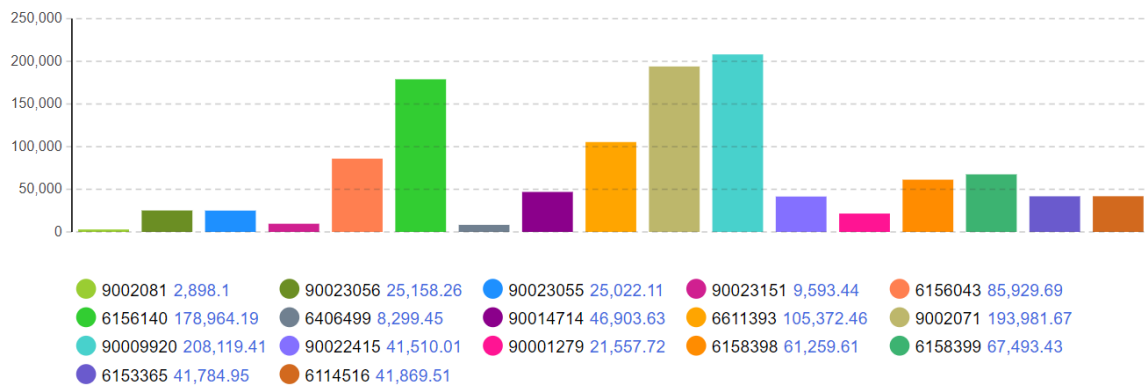


Abbildung 7.13: Darstellung der Zeit für jeden Auftrag

Im nächsten Schritt wird die Datenbank verändert und getestet, ob eine Beschleunigung der Zeit pro Auftrag erzielt wird. Da jede Abteilung in der Anzahl der Ressourcen begrenzt ist, fällt während der Simulation auf, dass es zu Wartezeiten kommt und Produkte warten müssen. Der erste Ansatz ist, dass zuerst Produkte gefertigt werden, die die geringste Anzahl an Arbeitsschritten benötigen.

1	0	90,022,415	100	0	0	0	0	0	0	0	140	0
2	1	9,002,081	20	48	0	0	0	0	0	0	0	0
3	2	6,406,499	15	0	0	30	0	0	0	0	0	12
4	3	6,153,365	20	6.8	6.67	0	0	0	0	0	0	0
5	4	90,001,279	20	8	6	0	0	0	0	0	0	0
6	5	6,114,516	30	40.2	5	0	0	0	0	0	0	0
7	6	90,023,056	13	26	13	0	26	39	0	0	0	0
8	7	90,023,055	13	26	13	0	26	39	0	0	0	0
9	8	90,014,714	20	76	24	0	0	0	0	0	0	60
10	9	6,158,398	25	47.5	0	0	0	300	0	0	30	0
11	10	6,158,399	25	47.5	0	0	0	375	0	0	30	0
12	11	90,023,151	13	0	0	0	78	156	130	0	0	0
13	12	6,156,043	30	90	78	0	0	600	0	0	0	102
14	13	9,002,071	120	0	0	0	0	1,560	0	0	540	960
15	14	90,009,920	60	252	210	0	0	2,400	0	0	0	300
16	15	6,611,393	398	704.46	159.2	0	0	0	0	0	0	796
17	16	6,156,140	108	255.96	108	0	0	1,944	0	0	0	302.4

Abbildung 7.14: Datenbank nach Veränderung der Reihenfolge

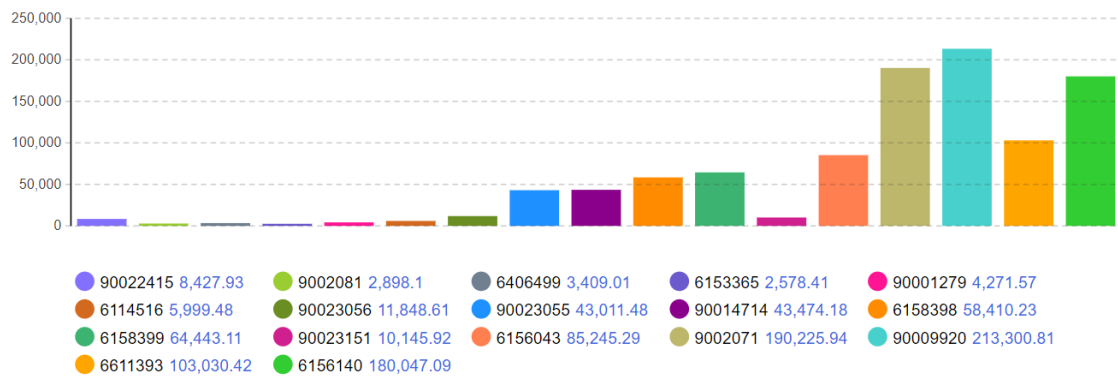


Abbildung 7.15: Darstellung der Zeit für jeden Auftrag nach Veränderung der Reihenfolge

In der *Abbildung 7.14* ist die Auswertung der Simulation zu sehen. Deutlich erkennbar sind die Aufträge, die die geringste Laufzeit benötigen. Der Vergleich dieser Aufträge weist lediglich eine minimale Verbesserung auf. Das Ergebnis dieser Aufträge ist nachvollziehbar, denn diese benötigen zwischen eins und zwei Arbeitsschritte. Demnach ist bei diesen Ergebnissen keine große Veränderung zu sehen.

Auffallend ist, dass die Gesamtlaufzeit nach dieser Veränderung ungefähr 217.000 Sekunden beträgt. Die unveränderte Datenbank erreicht eine Laufzeit von 211.860 Sekunden. Die Veränderung der Reihenfolge macht einen deutlichen Unterschied in der Laufzeit pro Auftrag aus. Sei es hinsichtlich einer Verbesserung oder Verschlechterung. Betrachtet man die blau markierte Artikelnummer in der *Abbildung 7.14* so ist eine deutliche Verbesserung der Laufzeit von diesem Auftrag zu sehen. So benötigt dieser in der standardmäßigen Reihenfolge eine Laufzeit von 41.784 Sekunden. Anhand der Veränderung der Reihenfolge wird dieser Auftrag in einer Zeit von 2.578 Sekunden gefertigt. Der Grund dafür ist, dass diese Aufträge kaum Arbeitsschritte benötigen. Wenn diese jedoch inmitten der Produktion gefertigt werden, so kommt es zu Wartezeiten. Das liegt daran, dass alle Maschinen aufgrund von begrenzten Ressourcen belegt sind und dafür diese in eine Warteschleife geraten und warten müssen, bis diese verarbeitet werden können.

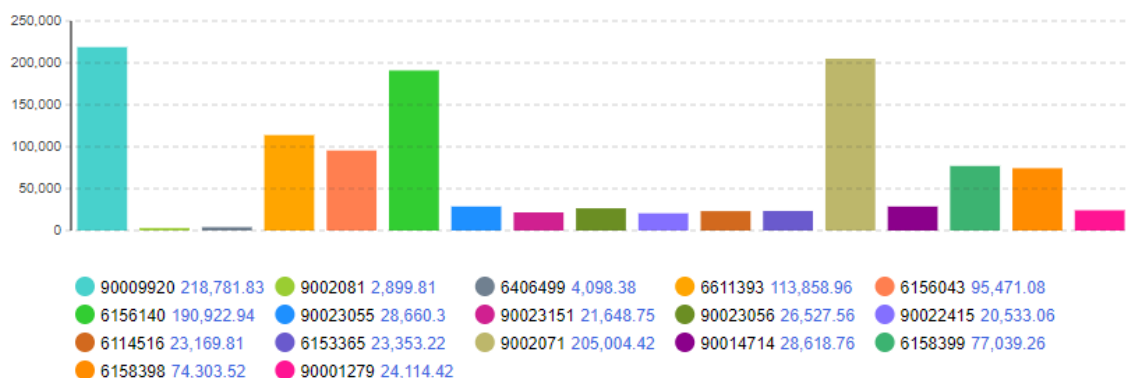


Abbildung 7.16: Gesamtlaufzeit zufällig generierter Reihenfolge: 220.391 Sekunden

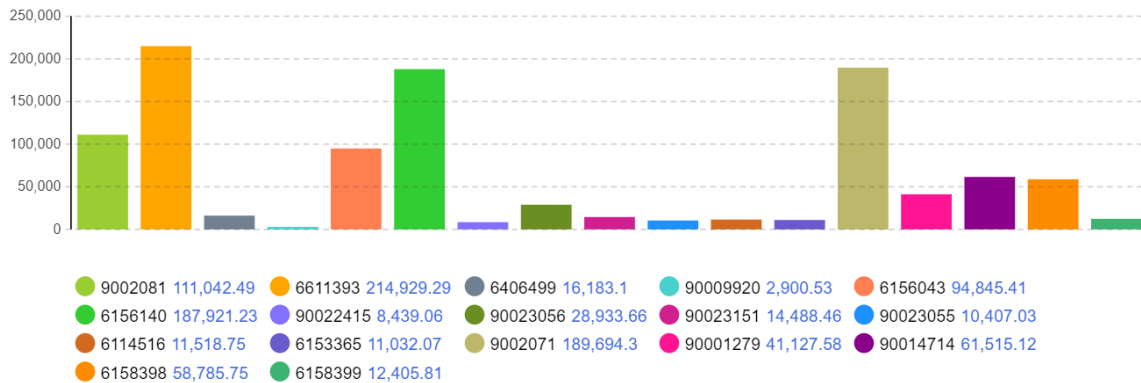


Abbildung 7.17: Gesamtlaufzeit zufällig generierter Reihenfolge: 216.614 Sekunden

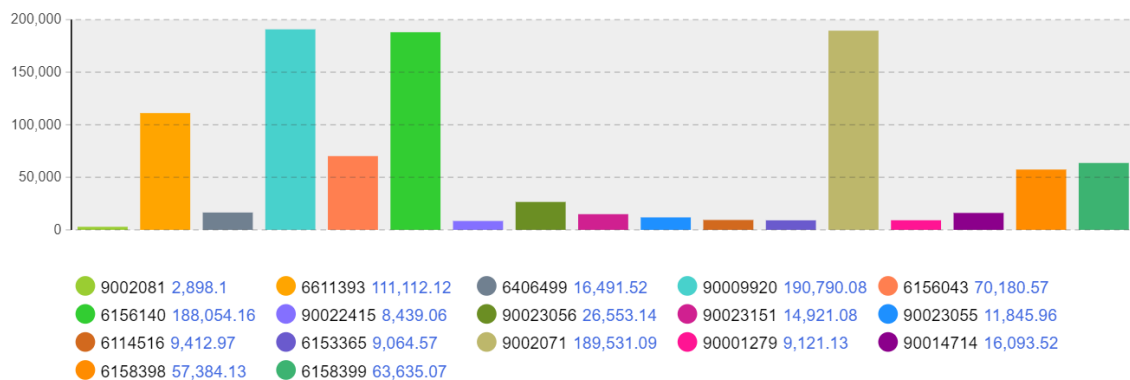


Abbildung 7.18: Gesamtlaufzeit zufällig generierter Reihenfolge: 206.891 Sekunden

Ein weiteres Beispiel ist das Erzeugen von zufälligen Reihenfolgen. Dafür wurde ein Programm in Python geschrieben, das ein Set in einer randomisierten Reihenfolge aus den verschiedenen Artikelnummern erzeugt. Nach einigen Testversuchen erreichte eine zufällige Reihenfolge (Abbildung 7.18) die geringste Gesamtlaufzeit. Diese beträgt 206.891 Sekunden. Vergleicht man das Ergebnis mit der unveränderten Reihenfolge aus der *Abbildung 7.14* so wird eine wesentliche Verbesserung erzielt. Ausschließlich durch die Veränderung der Reihenfolge wird eine Zeitersparnis von 1,383 Stunden generiert.

8 Zusammenfassung

In diesem Kapitel handelt es sich um die Zusammenfassung und Fazit der Arbeit und der Simulationssoftware AnyLogic. Die Simulationssoftware ist anhand der Erweiterung mit Java ein sehr komplexes Tool. Einerseits ermöglicht AnyLogic dem User komplexe Simulationen zu erstellen, wie zum Beispiel einen Produktionsablauf mit einer zusätzlichen Datenbank. Zudem bietet es einem Programmierer, der Kenntnisse in der Programmiersprache Java hat, Probleme leichter zu erkennen und zu beheben. Andererseits ist das Lösen von Problemen sehr zeitintensiv und Ideen sind nicht immer einfach umzusetzen.

Zu Beginn der Arbeit und somit auch der Simulation waren die Probleme recht gering und Probleme beanspruchten wenig Aufwand zur Behebung. Im Laufe der Zeit wurde die Simulation immer größer, realistischer und infolgedessen deutlich komplexer. Das Einsetzen von Objekten oder Agenten stellte meist keinerlei Probleme dar. Nach dem Verwenden von Funktionen, Datenbanken und Abfragen wurde die Simulation deutlich aufwendiger und anfälliger für Fehler. Für das Erreichen von spezifischen Zielen oder Meilensteinen in dieser Arbeit wurde immer mehr Zeit beansprucht. Die Erstellung der Simulation hat im Vergleich zur schriftlichen Ausarbeitung deutlich mehr Zeit in Anspruch genommen.

AnyLogic ermöglicht dem Nutzer eine realistische Simulation zu erstellen. Dies geschieht vor allem anhand der Verknüpfung mit der Programmiersprache Java. Zudem wäre die Erstellung einer solchen Simulation durch die Verwendung der Maus nicht möglich. Die Verwendung von Java ermöglicht es komplexe Simulationen zu erstellen.

Anhand der Möglichkeiten und des Spektrums, die die Software bietet, ist der User in der Lage eine große Anzahl an Problemen und Systemen zu simulieren.

Außerdem ist ein wesentlicher Vorteil an der gefertigten Simulation, dass diese für einen Großteil an Produktionssystemen anwendbar ist. Das bedeutet, dass die Funktionen, Parameter, Variablen, Datenbanken und Logik minimal verändert werden müssen, um diese für ein anderes System anzuwenden.

Die Analyse des Produktionsflusses zeigt deutlich, dass das zu simulierende Unternehmen keine optimale Auslastung der Maschinen und Abteilungen aufweist. Das ist für das Unternehmen von Nachteil, denn diese verfolgt die Absicht, dass die Maschinen und Abteilungen eine maximale Auslastung aufweisen. Der Ansatz des Unternehmens für den Bau eines Zentrallagers erbringt keine Zeitersparnis in der Gesamtlaufzeit. Dies würde wiederum bedeuten, dass der Bau weder die Effizienz noch die Geschwindigkeit des Produktionsflusses optimiert. Jedoch kann der Bau eines Zentrallagers nicht messbare Vorteile generieren. Die wären die Zentralisierung von Abläufen und eine hohe Lieferbereitschaft. Diese würden es dem Unternehmen ermöglichen, aufgrund von verfügbaren Lagerbeständen, dass Artikel stets lieferbar sind.

Eine Annahme war, dass die Veränderung der Reihenfolge von Aufträgen zu einer Verbesserung führen könnte. In einigen Fällen erwies sich diese Annahme als wahr. Eine Reihenfolge erledigte die Aufträge in einer Gesamtlaufzeit von 206.891 Sekunden. Dadurch wurde eine

Zeitersparnis von 1,383 Stunden generiert. Jedoch führten auch einige Reihenfolgen zu einer Verschlechterung der Gesamtlaufzeit.

Als letztes werden die Vor- und Nachteile der Simulationssoftware durch die gesammelten Erfahrungswerte gegenübergestellt. Die Simulation ermöglicht die Erstellung von komplexen Simulationen. Java Kenntnisse haben mir die Arbeit deutlich erleichtert. Zudem trägt die Verwendung der Programmiersprache zur Auffrischung und Verbesserung der Kenntnisse bei. Das Testen der verschiedenen Konstellationen von Parametern und Umstrukturierung des Produktionsablaufes verringert das Risiko. Simulationen nehmen in der aktuellen Zeit einen großen Platz ein. Die Gründe dafür wurden in der Arbeit ausgiebig erläutert. Simulationen werden in der Zukunft ebenfalls zum Einsatz kommen, um Risiken zu minimieren und Optimierungsansätze zu testen.

9 Referenzen

- [1] <https://www.geeksforgeeks.org/comparison-centralized-decentralized-and-distributed-systems/>, Parikshit Hooda, Stand: 2.Feb.2022/11:40
- [2] <https://www.ip-insider.de/was-ist-das-peer-to-peer-modell-a-943950/>, Dipl.-Ing. (FH) Stefan Luber / Dipl.-Ing. (FH) Andreas Donne, 3.Feb.2022/16:18
- [3] <https://www.ip-insider.de/was-ist-das-master-slave-modell-a-943967/>, Dipl.-Ing. (FH) Stefan Luber / Dipl.-Ing. (FH) Andreas Donne, 4.Feb.2022/8:18
- [4] <https://de.wikipedia.org/wiki/Peer-to-Peer#/media/Datei:P2P-network.svg>
- [5] Organic Computing II, Chapter 5: Optimisation, Prof. Dr. Jörg Hähner (Universität Augsburg), Prof. Dr. Sven Tomforde (Universität Kassel), Prof. Dr. Müller-Schloer (Universität Hannover)
- [6] https://www.splunk.com/en_us/data-insider/what-are-distributed-systems.html, Stand: 6.Feb.2022, 18:15
- [7] An Introduction to Optimization, Edwin K. P. Chong, Stanislaw H. Zak
- [8] Optimization: Algorithms and Applications, Rajesh Kumar Arora
- [9] Nature Inspired Based Meta-heuristic Techniques for Global Applications, Rachhpal Singh, P.G. Department of Computer Science and Applications, Khalsa College, Amritsar (INDIA)
- [10] TABU SEARCH, Fred Glover, Manuel Laguna
- [11] AnyLogic Homepage, <https://www.anylogic.de/>, <https://anylogic.help/>
- [12] Introduction to modeling and simulation, Anu Maria
- [13] Kanban: Optimale Steuerung von Prozessen, Gerhard Geiger, Ekbert Hering, Rolf Kummer
- [14] Metaheuristic Algorithms: A Comprehensive Review, Mohamed Abdel-Basset, Laila Abdel-Fatah, Arun Kumar Sangaiah, Computational Intelligence for Multimedia Big Data on the Cloud with Engineering, 2018
- [15] <https://www.enzyklopaedie-der-wirtschaftsinformatik.de/lexikon/informationssysteme/SektorspezifischeAnwendungssysteme/Produktionsplanungs-und-steuerungssystem/Zeitwirtschaft>, Stand: 28.März.2022, Prof. Dr. Christoph Siepermann, Hochschule Albstadt-Sigmaringen
- [16] <https://de-academic.com/dic.nsf/dewiki/165513>, Stand: 29.03.2022, 13:32
- [17] AnyLogic Software
- [18] Computational Complexity: A Modern Approach, Sanjeev Arora, Boaz Barak, Princeton University

- [19] <https://wirtschaftslexikon.gabler.de/definition/system-50117>, Prof. Dr. Robert Gillenkirch Universität Osnabrück, Stand: 13.04.2022
- [20] Ad-hoc and Sensor Networks, Time Synchronisation, Prof. Dr. Jörg Hähner, Universität Augsburg
- [21] <https://de.wikipedia.org/wiki/Kanban>, Stand: 17.Mai.2022, 20:04
- [22] Ad-hoc and Sensor Networks, Introduction, Prof. Dr. Jörg Hähner, Universität Augsburg

A Anhang

Dinge, die sonst nirgends Platz fanden oder einfach noch erwähnt werden müssen.

B Inhalt der CD

Im Anhang dieser Arbeit befindet sich eine CD mit folgendem Inhalt:

- Diese Arbeit als PDF Datei - im Ordner *PDF*
- Der Sourcecode der Implementierung dieser Arbeit - im Ordner *SRC*
- Die Implementierung als .jar-Datei - im Ordner *JAR*
- Den $\text{\LaTeX}$ -Sourcecode - im Ordner *LATEX*

C Erklärung

Ich erkläre hiermit, dass ich die Arbeit eigenhändig, ohne unzulässige Hilfsmittel und unter Angabe aller Quellen angefertigt habe.

Augsburg, den 23. Mai 2022

Onurcan Ünsal